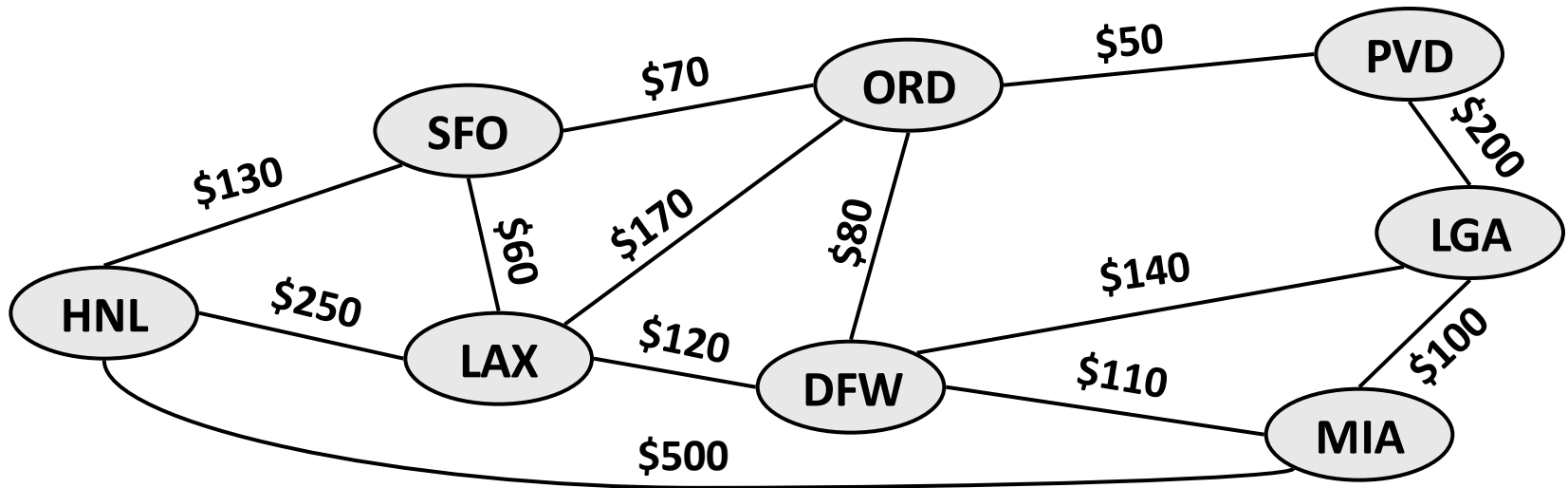


Design and Analysis of Algorithm

Topic: Graph Algorithms

Searching for paths

- Searching for a path from one vertex to another:
 - Sometimes, we just want *any* path (or want to know there *is* a path).
 - Sometimes, we want to minimize path *length* (# of edges).
 - Sometimes, we want to minimize path *cost* (sum of edge weights).



Finding Paths

- Easiest way: Depth-First Search (DFS)
 - Recursive backtracking!
- Finds a path between two nodes if it exists
 - Or can find all the nodes **reachable** from a node
 - Where can I travel to starting in San Francisco?
 - If all my friends (and their friends, and so on) share my post, how many will eventually see it?

Depth-First Search

- **Depth-First Search (DFS):** Finds a path between two vertices by exploring each possible path as far as possible before backtracking.
 - Often implemented recursively.
 - Many graph algorithms involve *visiting* or *marking* vertices.

- DFS from *a* to *h* (assuming A-Z order) visits:

- a

- b

- e

- f

- c

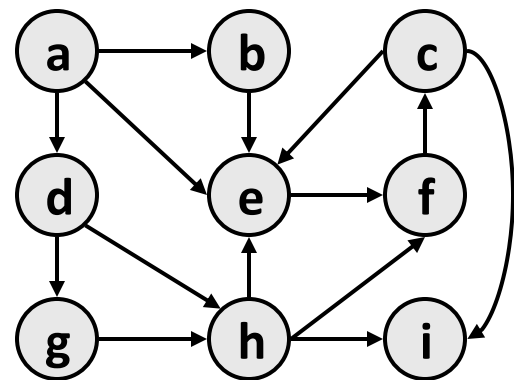
- i

- d

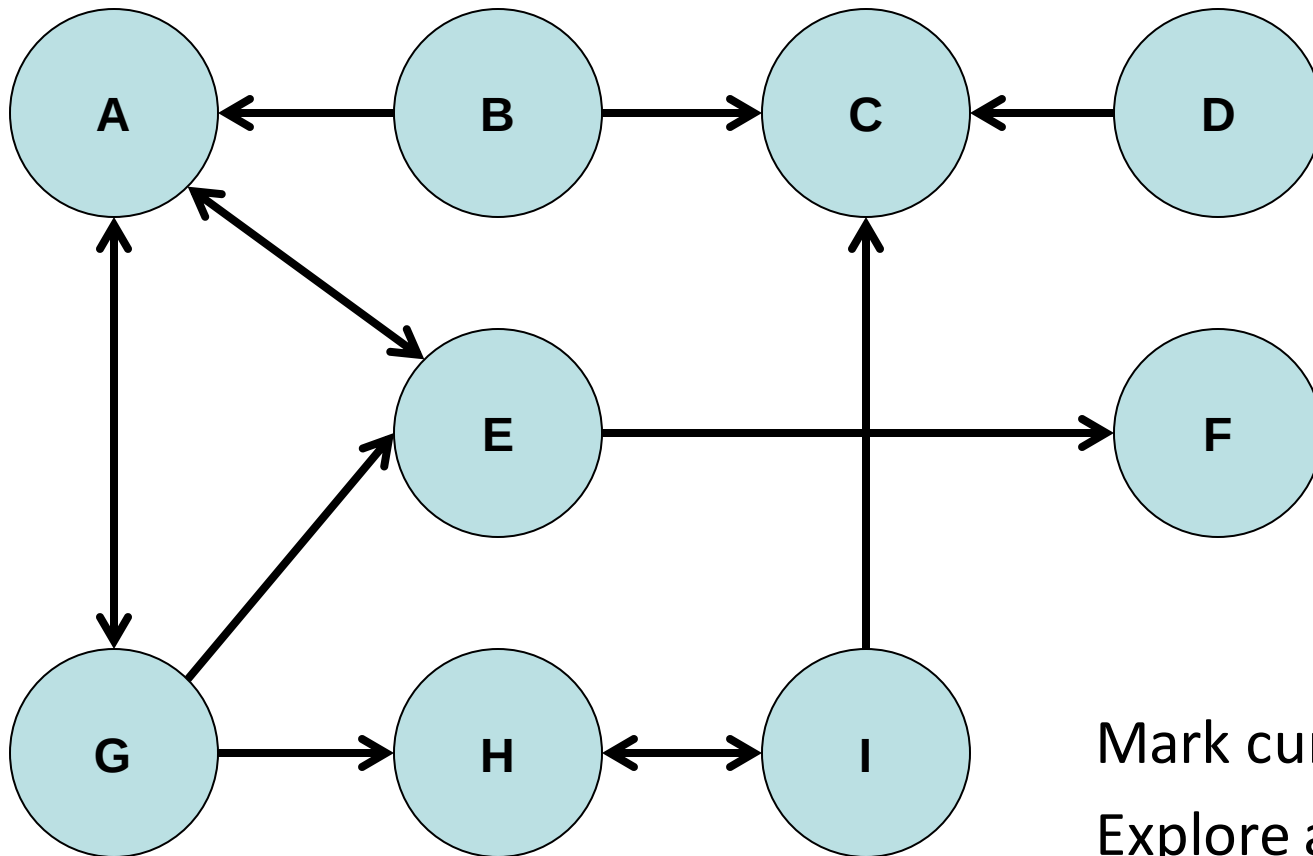
- g

- h

- path found: {a, d, g, h}

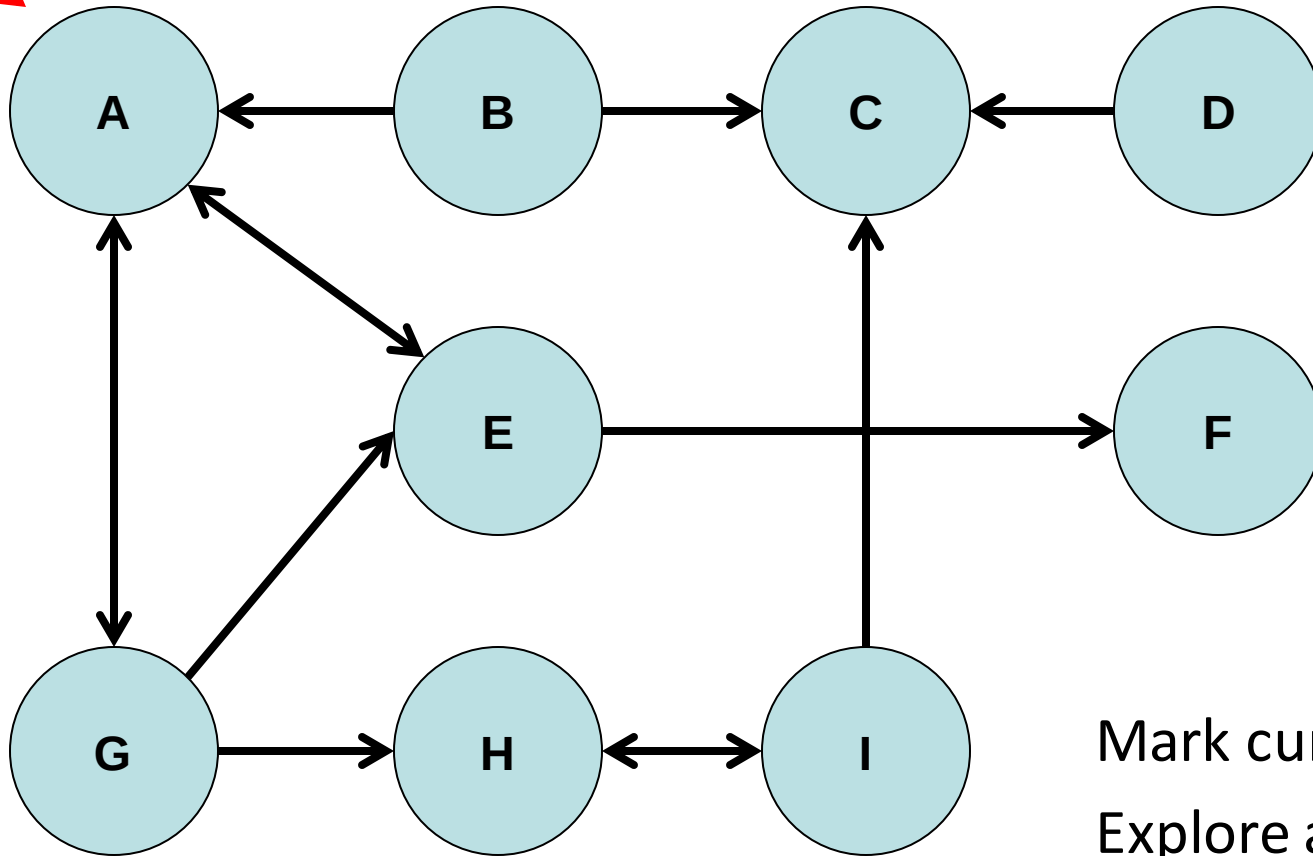


DFS



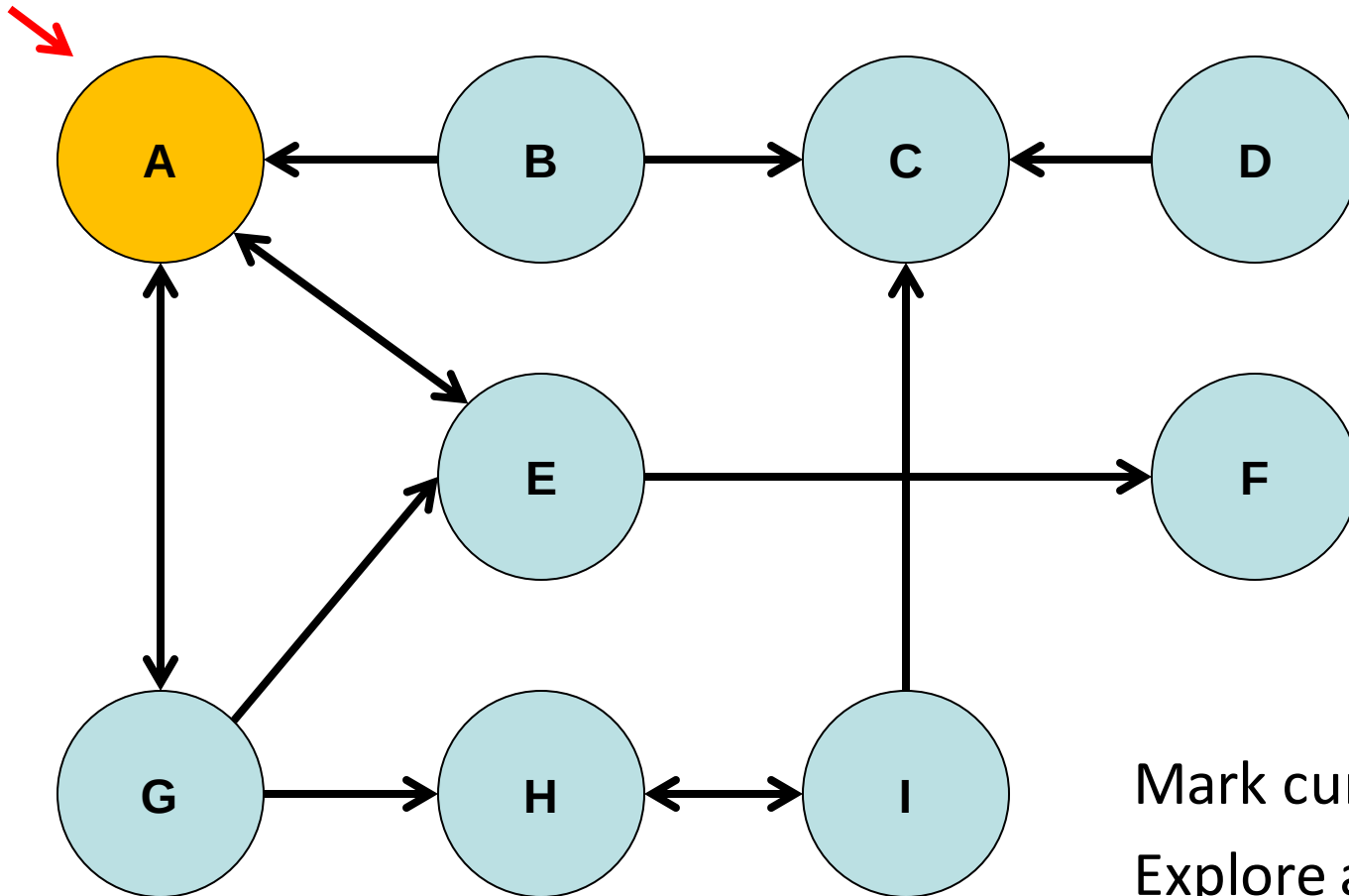
Mark current as visited
Explore all the unvisited
nodes from this node

DFS



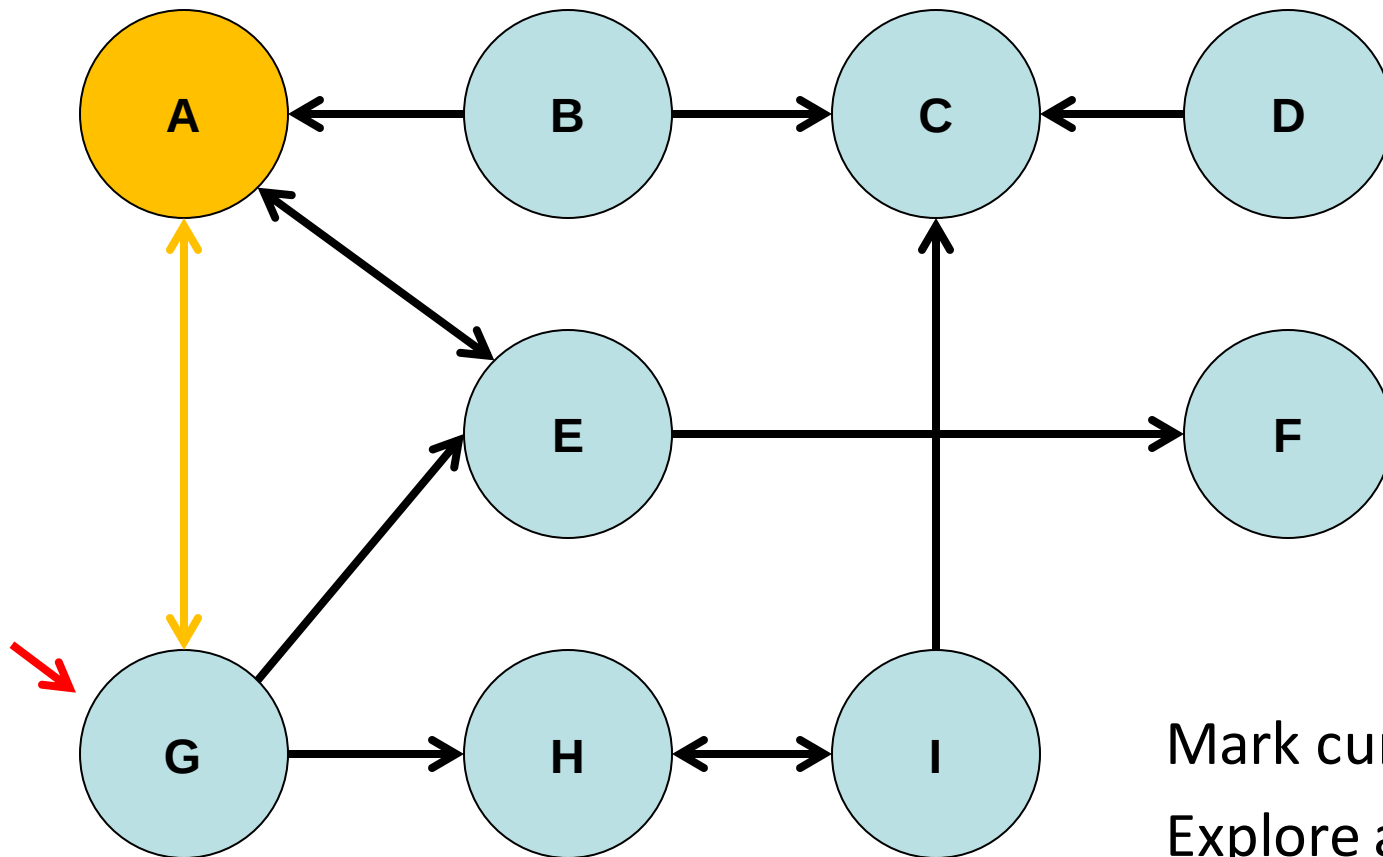
Mark current as visited
Explore all the unvisited
nodes from this node

DFS



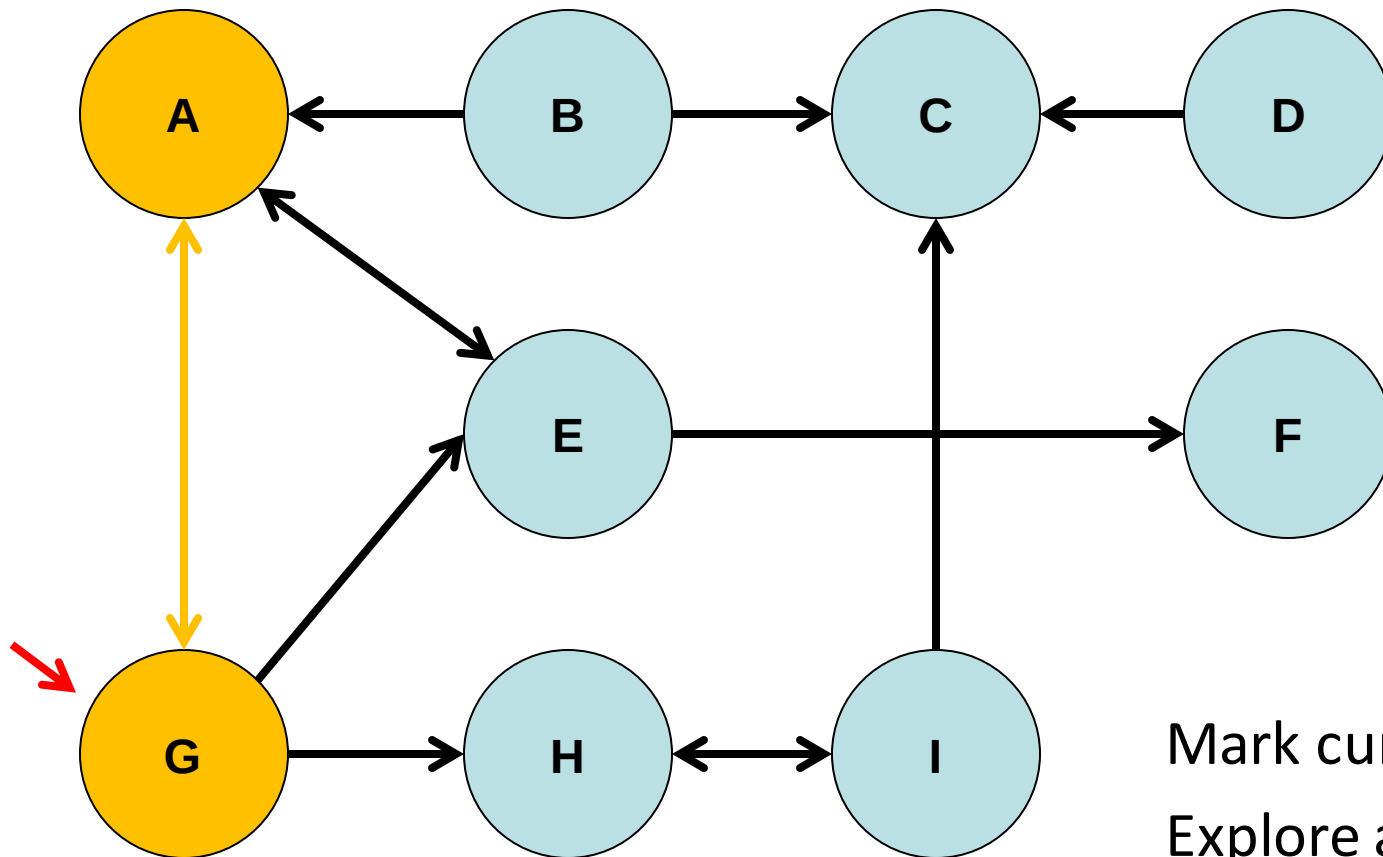
Mark current as visited
Explore all the unvisited
nodes from this node

DFS



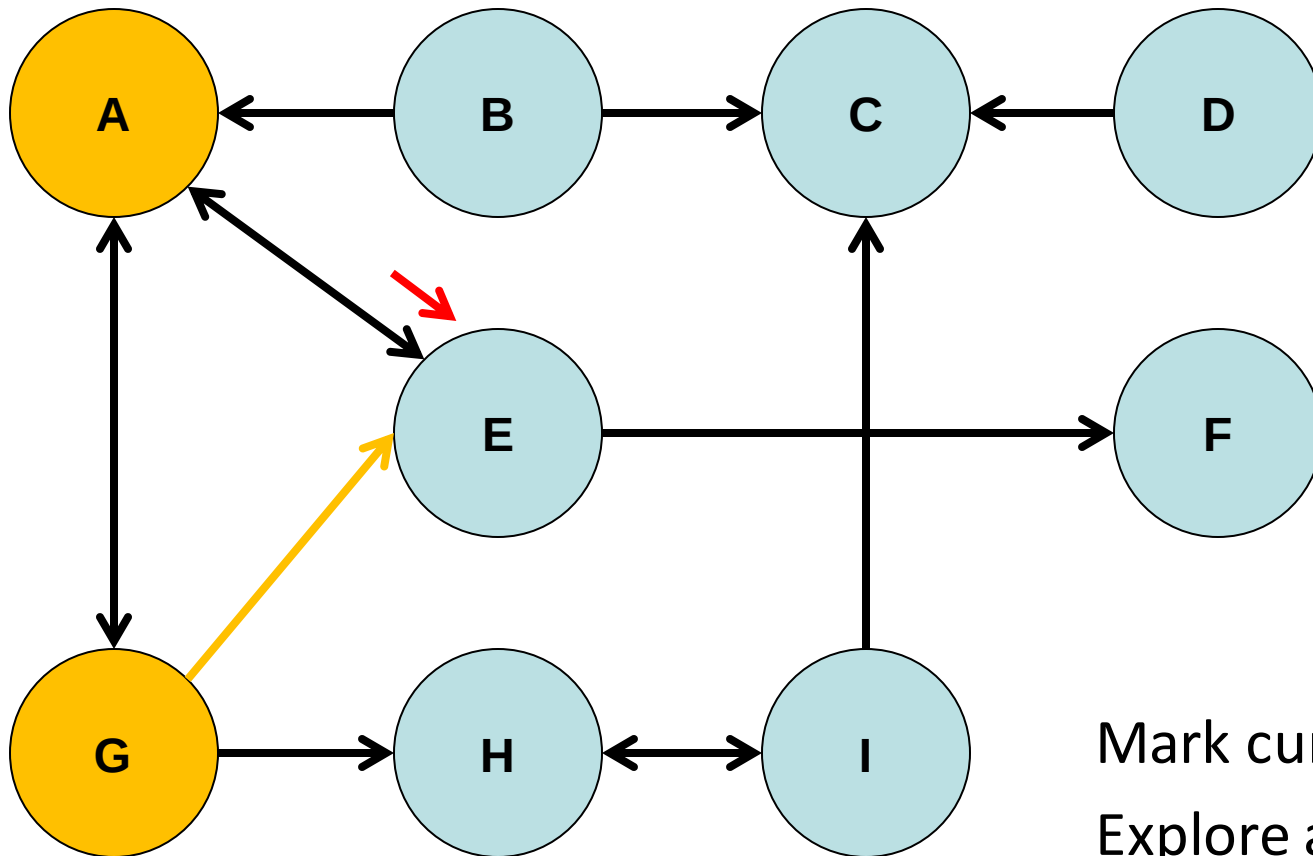
Mark current as visited
Explore all the unvisited
nodes from this node

DFS



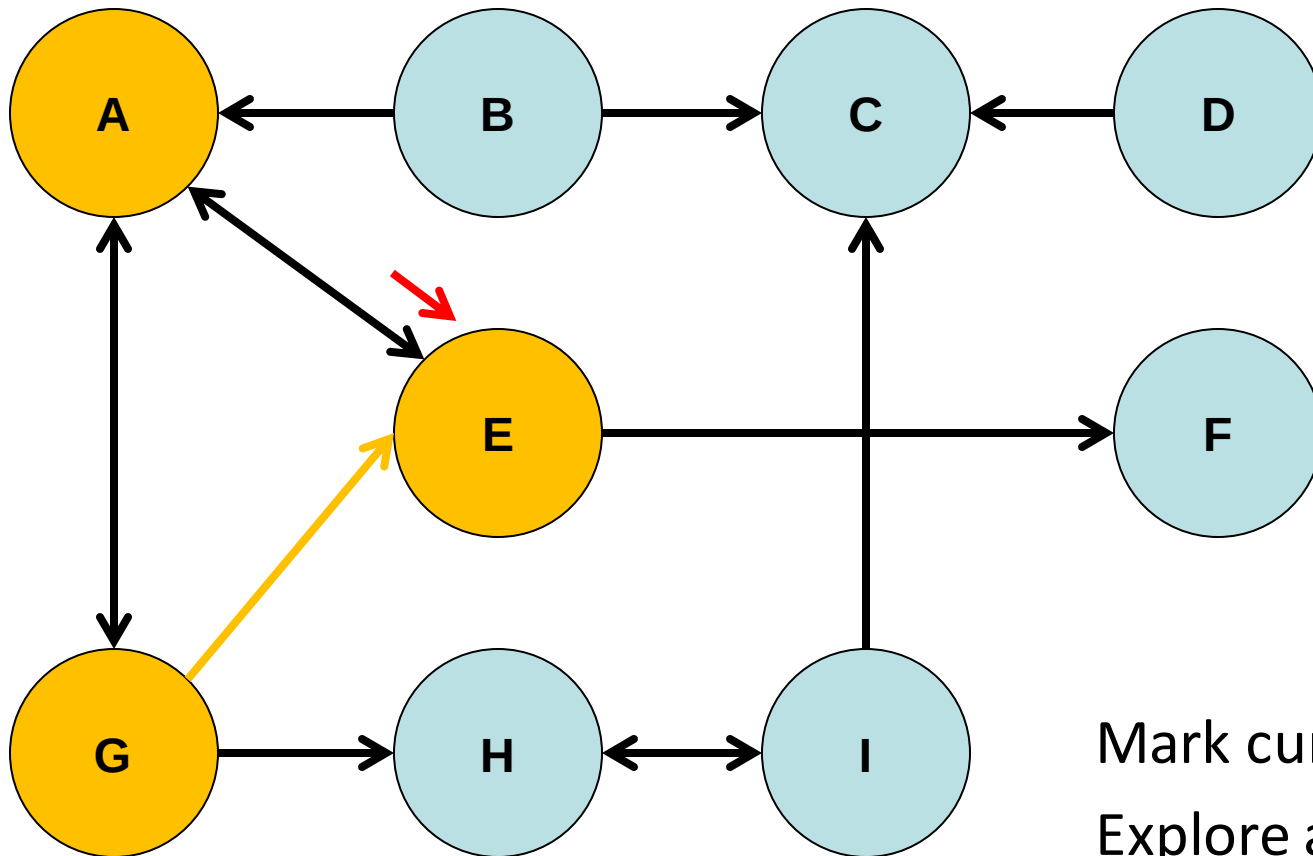
Mark current as visited
Explore all the unvisited
nodes from this node

DFS



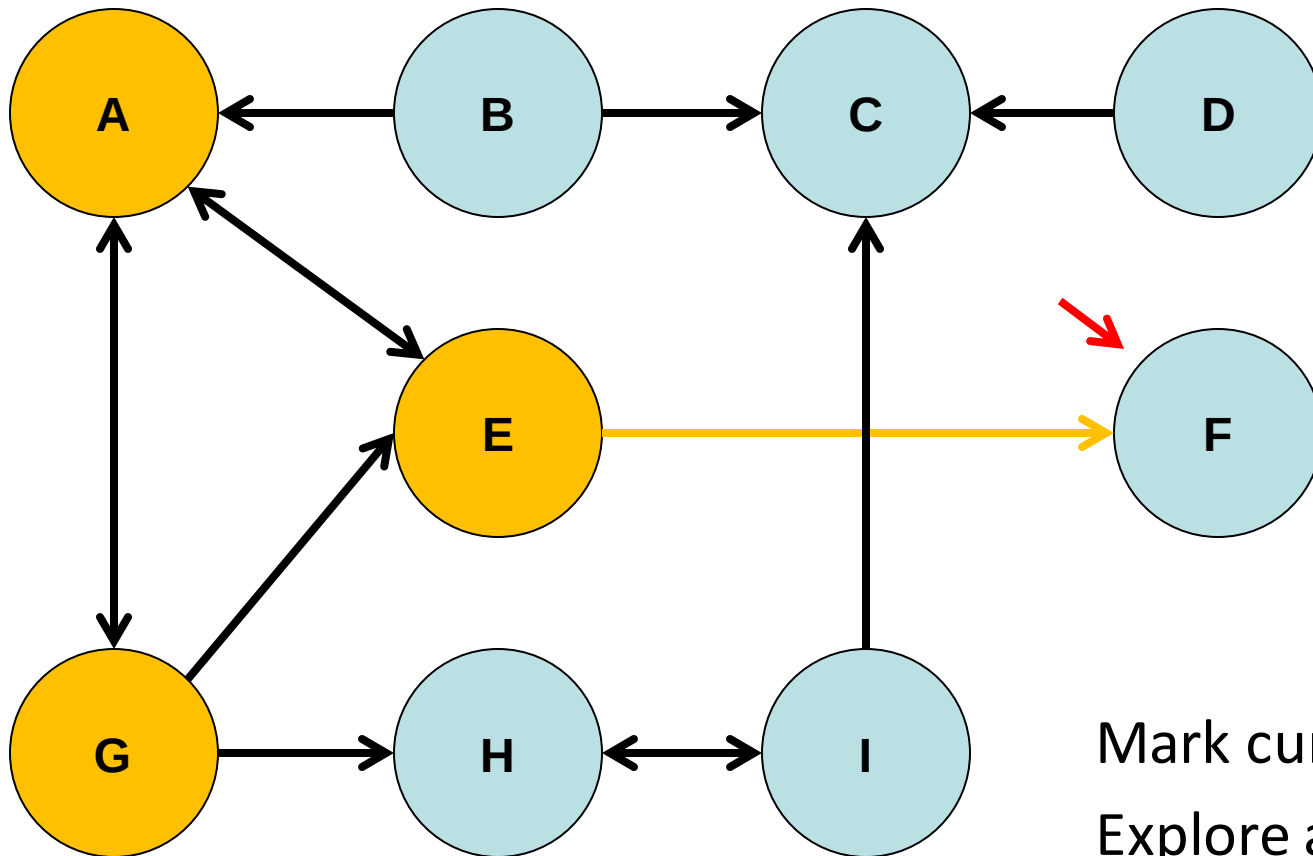
Mark current as visited
Explore all the unvisited
nodes from this node

DFS



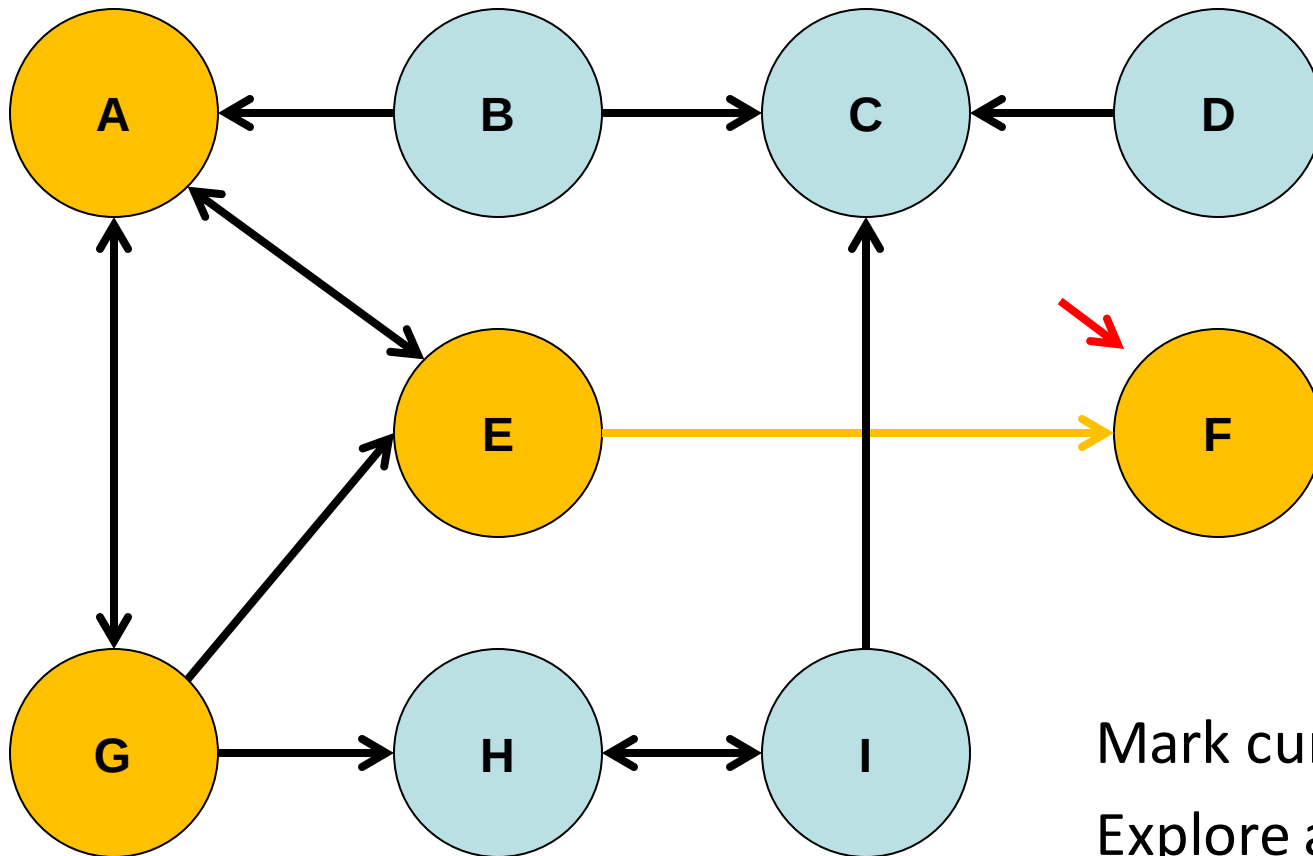
Mark current as visited
Explore all the unvisited
nodes from this node

DFS



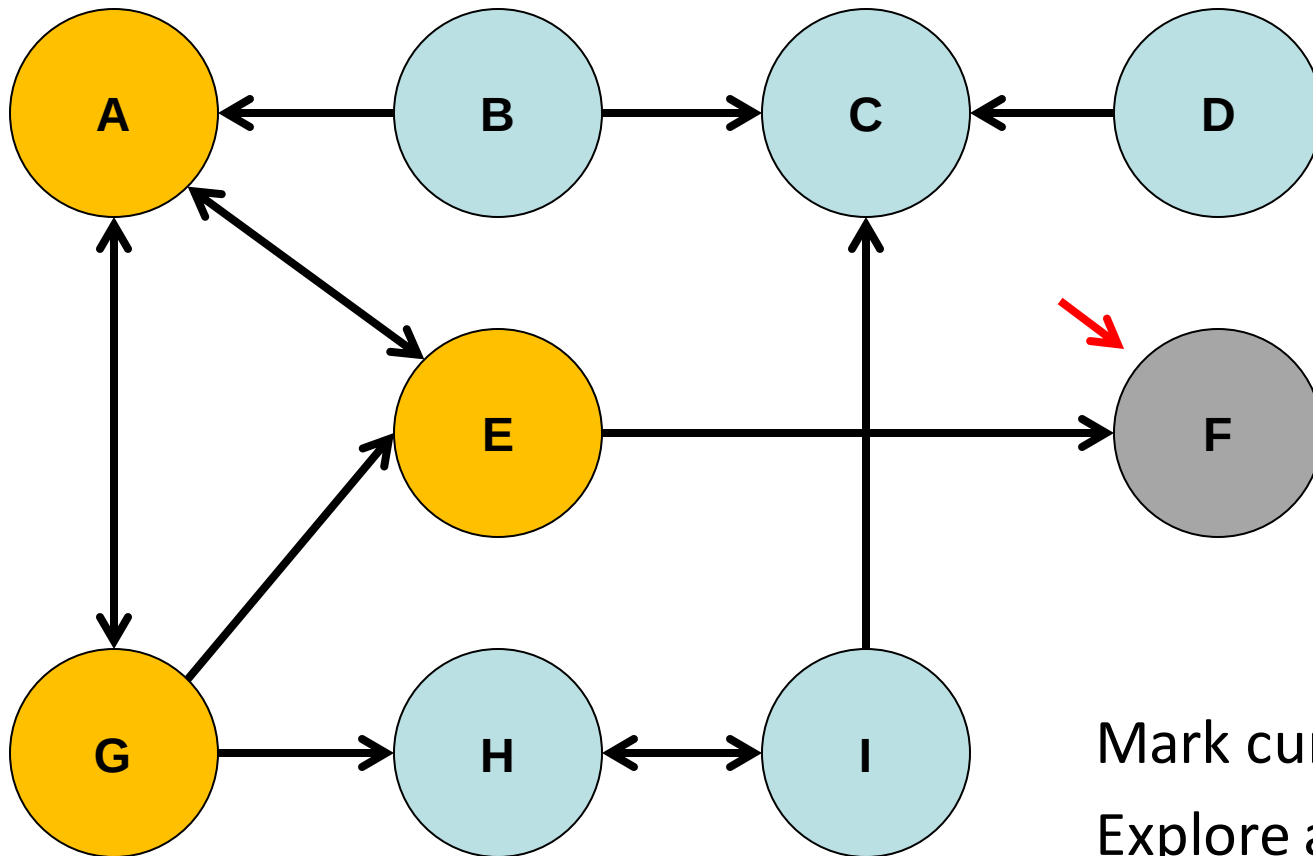
Mark current as visited
Explore all the unvisited
nodes from this node

DFS



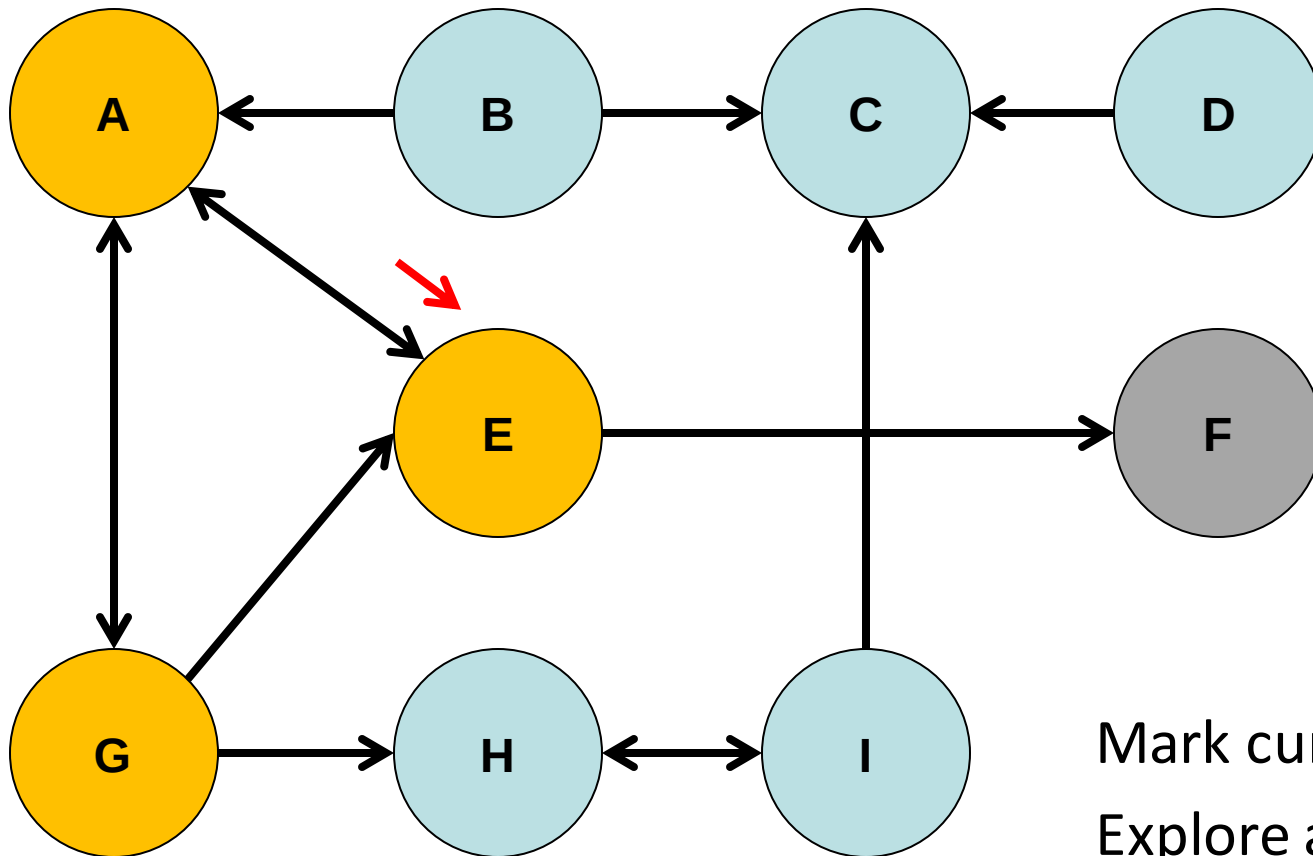
Mark current as visited
Explore all the unvisited
nodes from this node

DFS



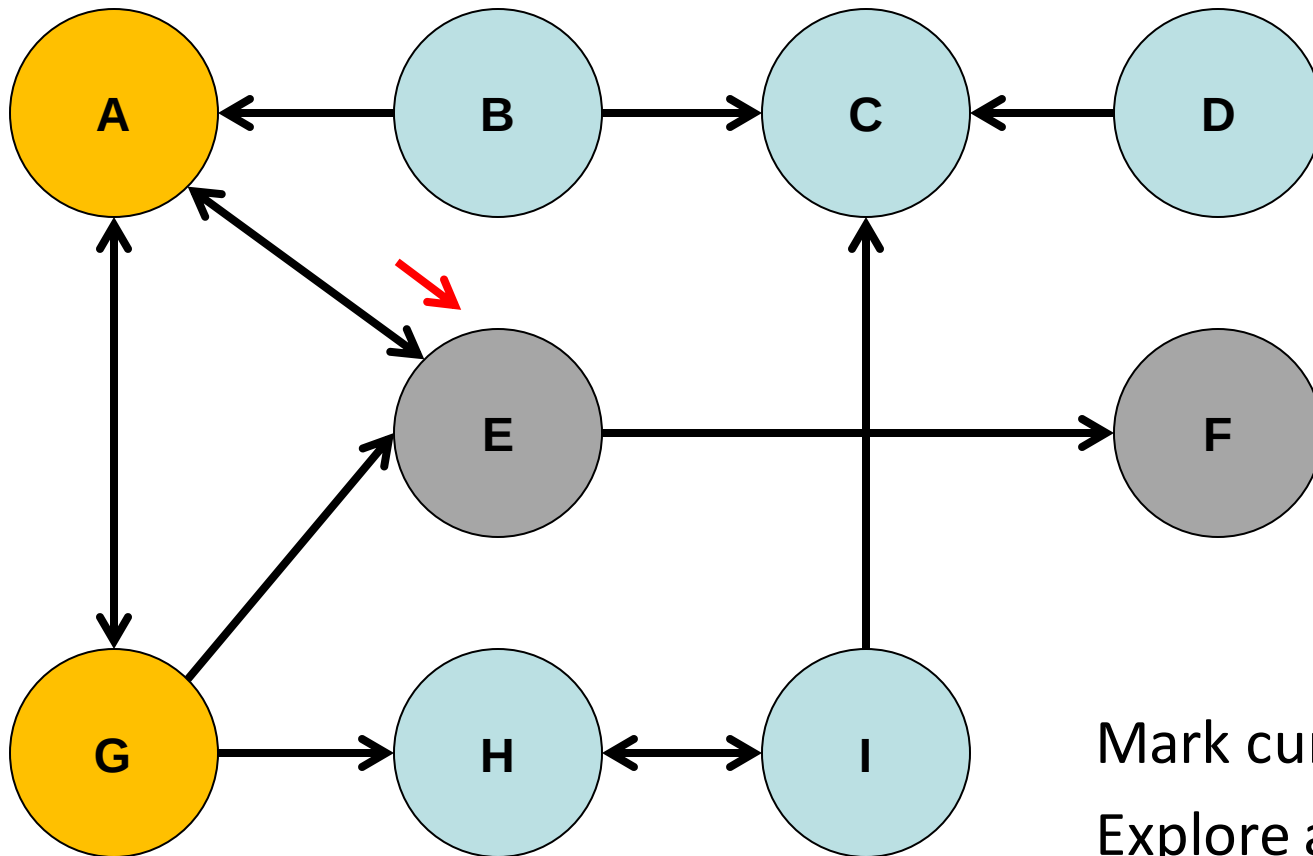
Mark current as visited
Explore all the unvisited
nodes from this node

DFS



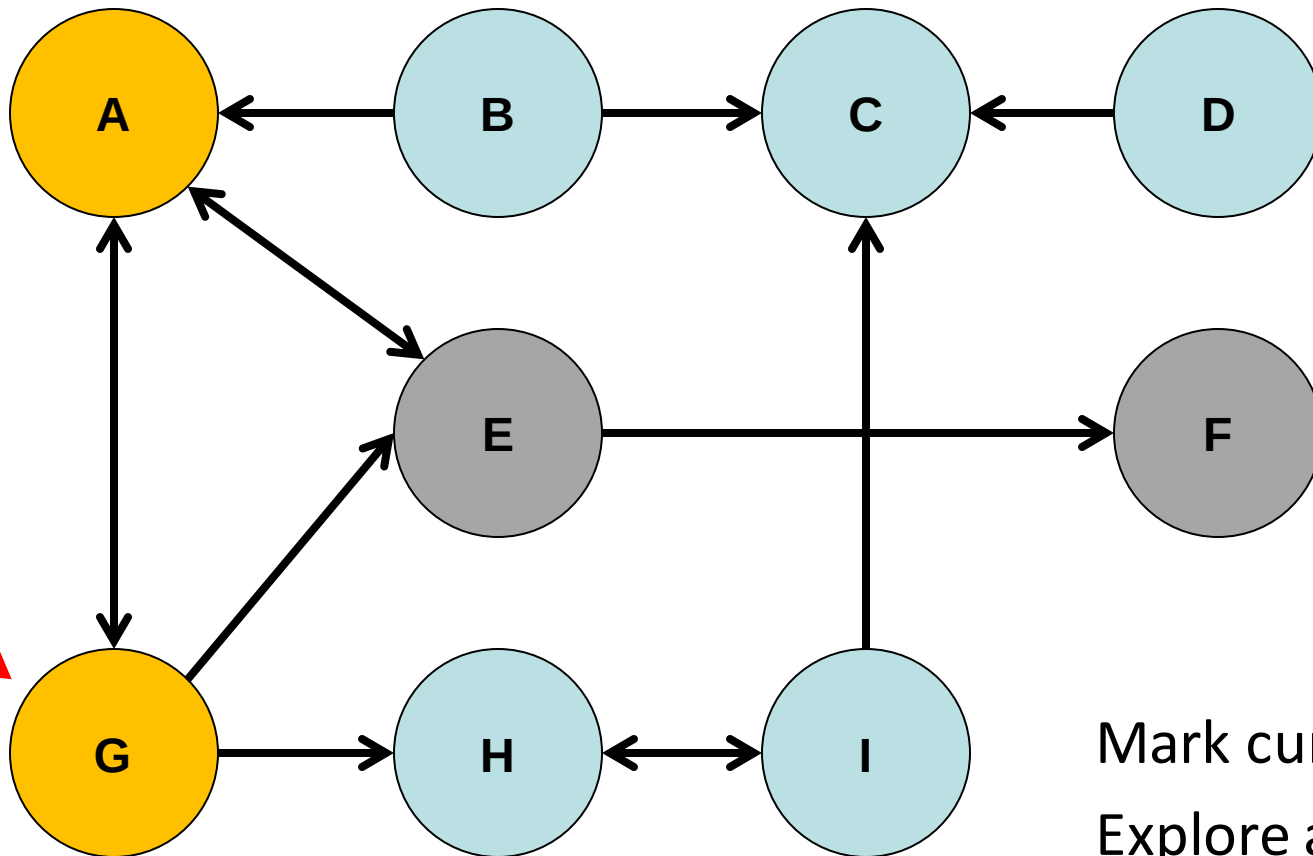
Mark current as visited
Explore all the unvisited
nodes from this node

DFS



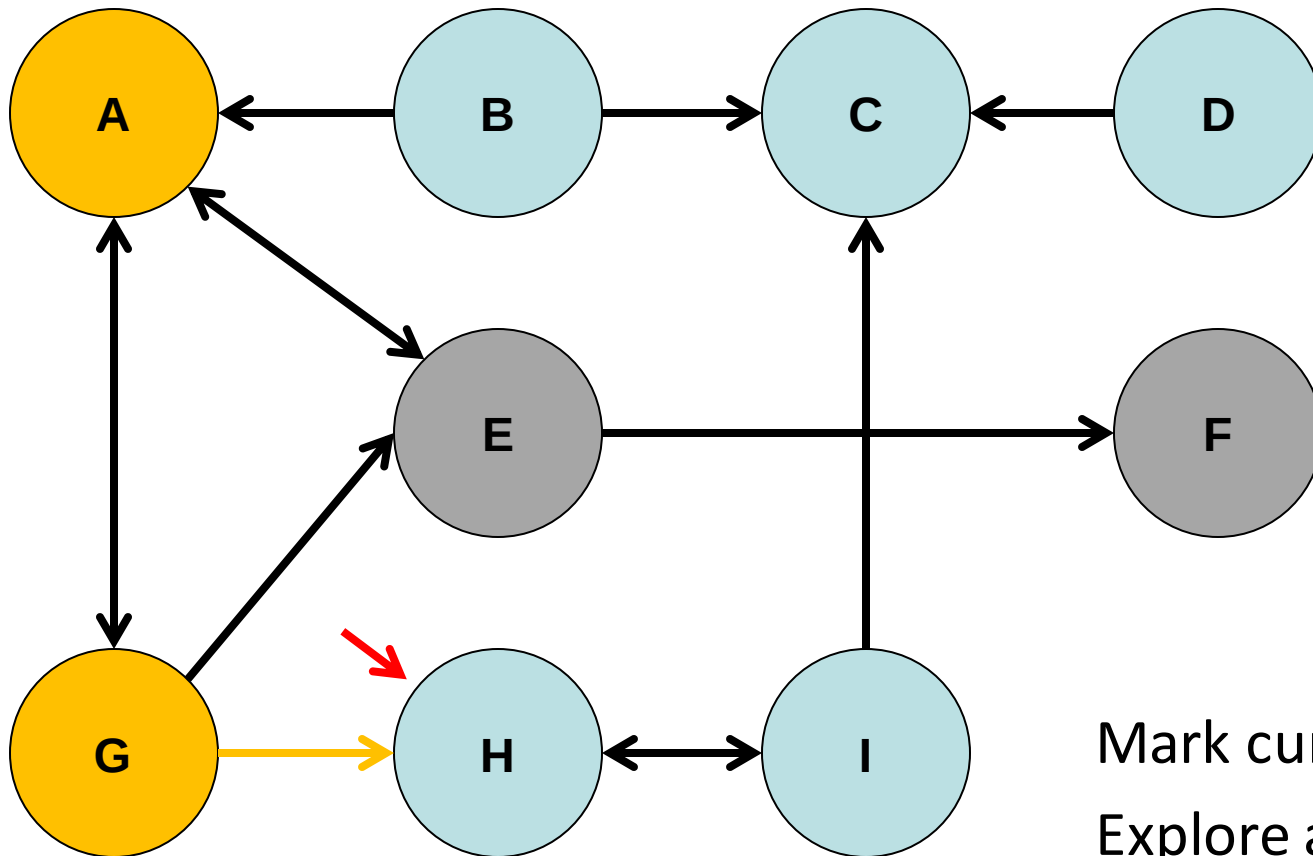
Mark current as visited
Explore all the unvisited
nodes from this node

DFS



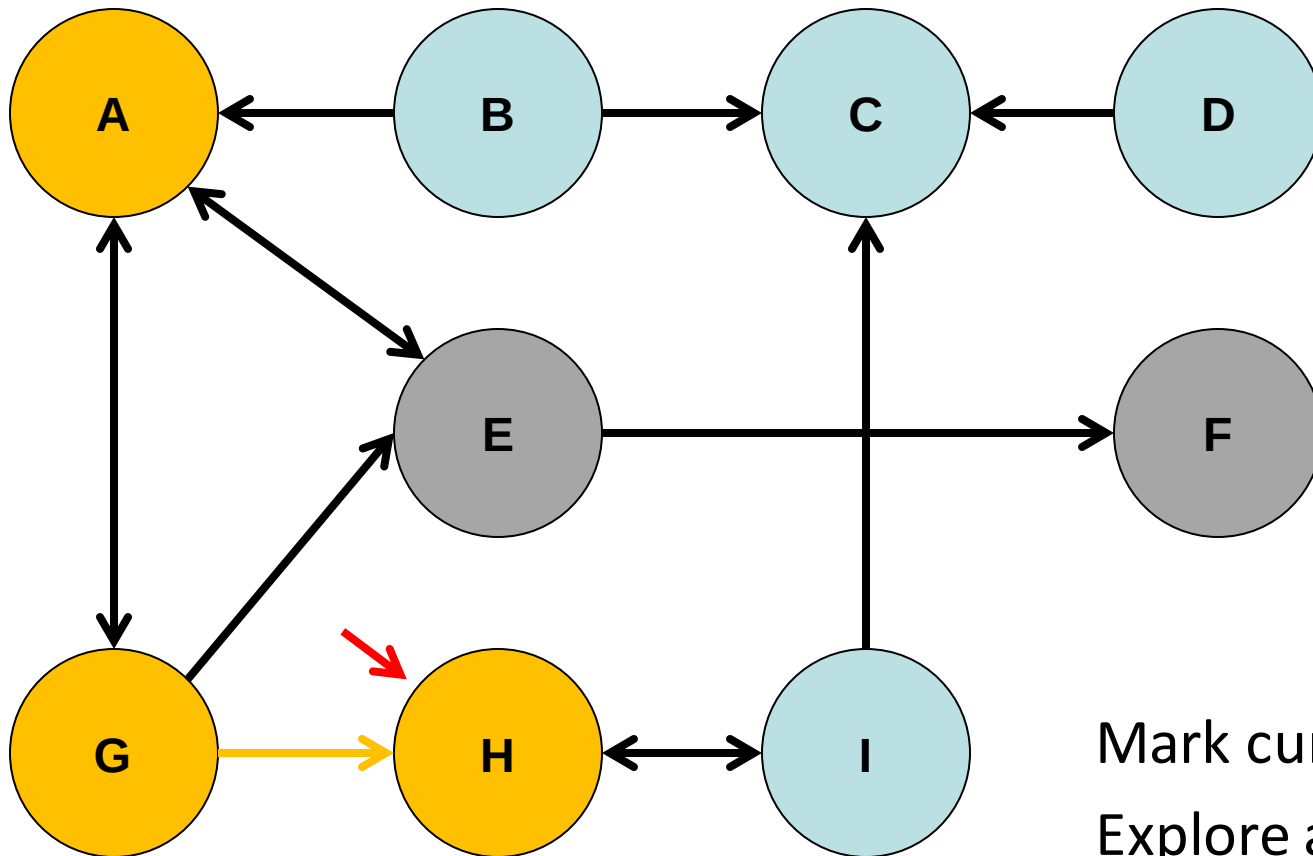
Mark current as visited
Explore all the unvisited
nodes from this node

DFS



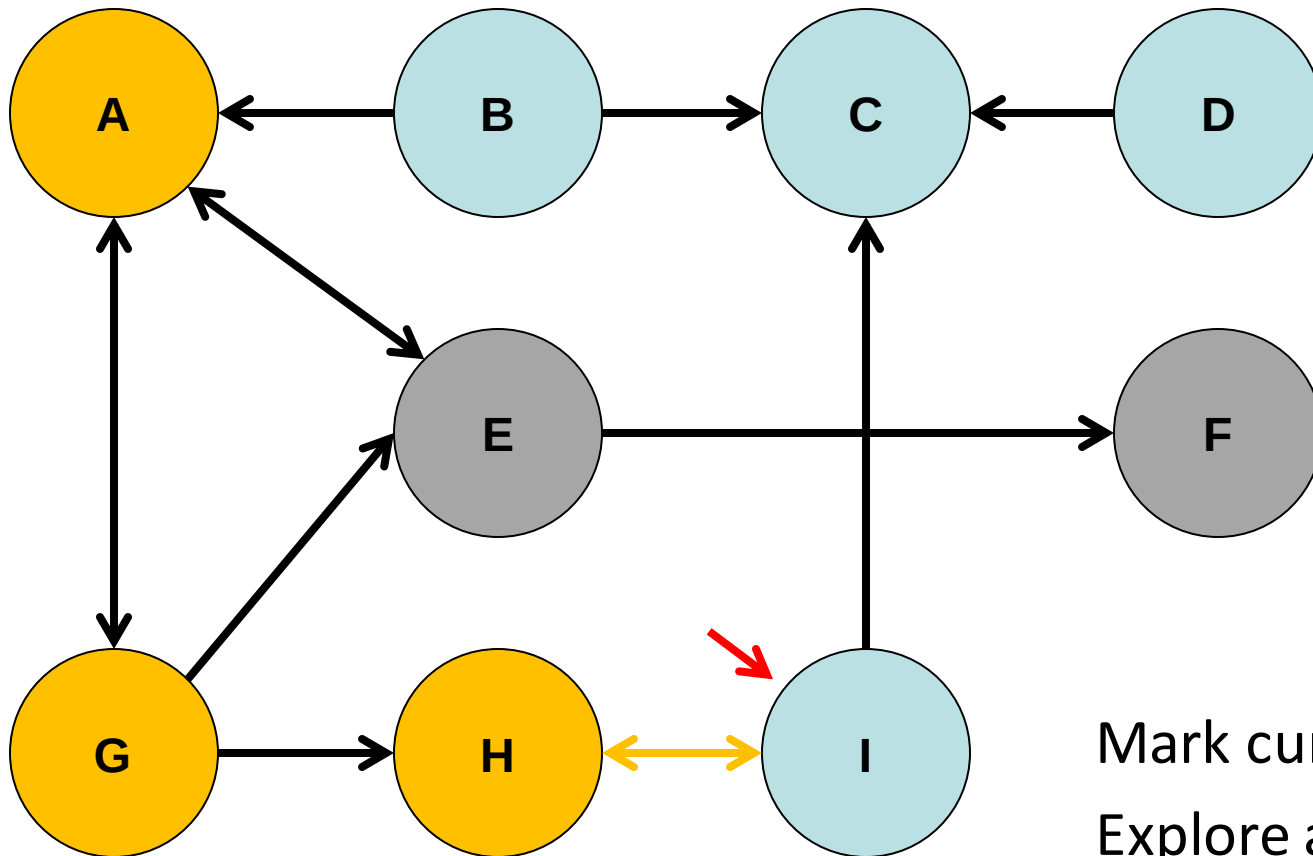
Mark current as visited
Explore all the unvisited
nodes from this node

DFS



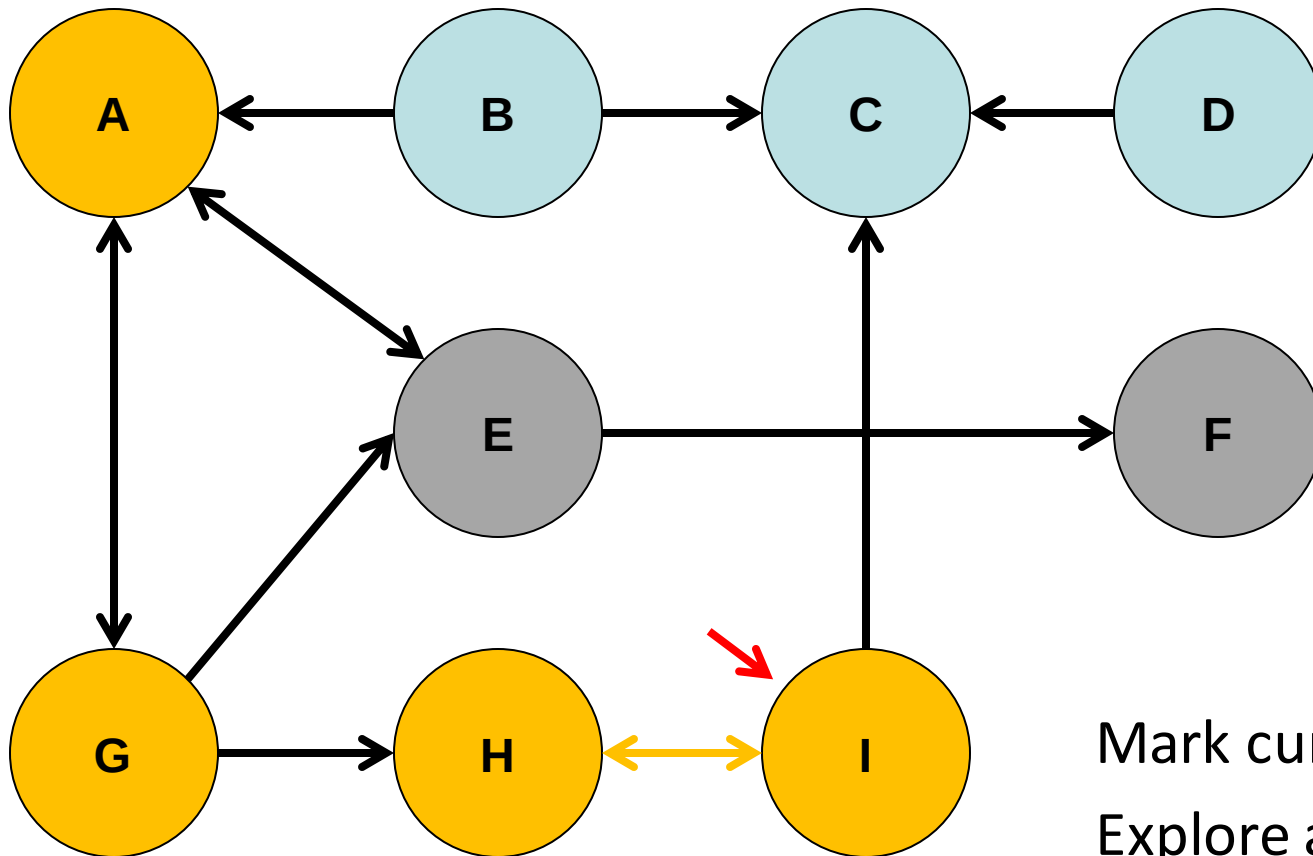
Mark current as visited
Explore all the unvisited
nodes from this node

DFS



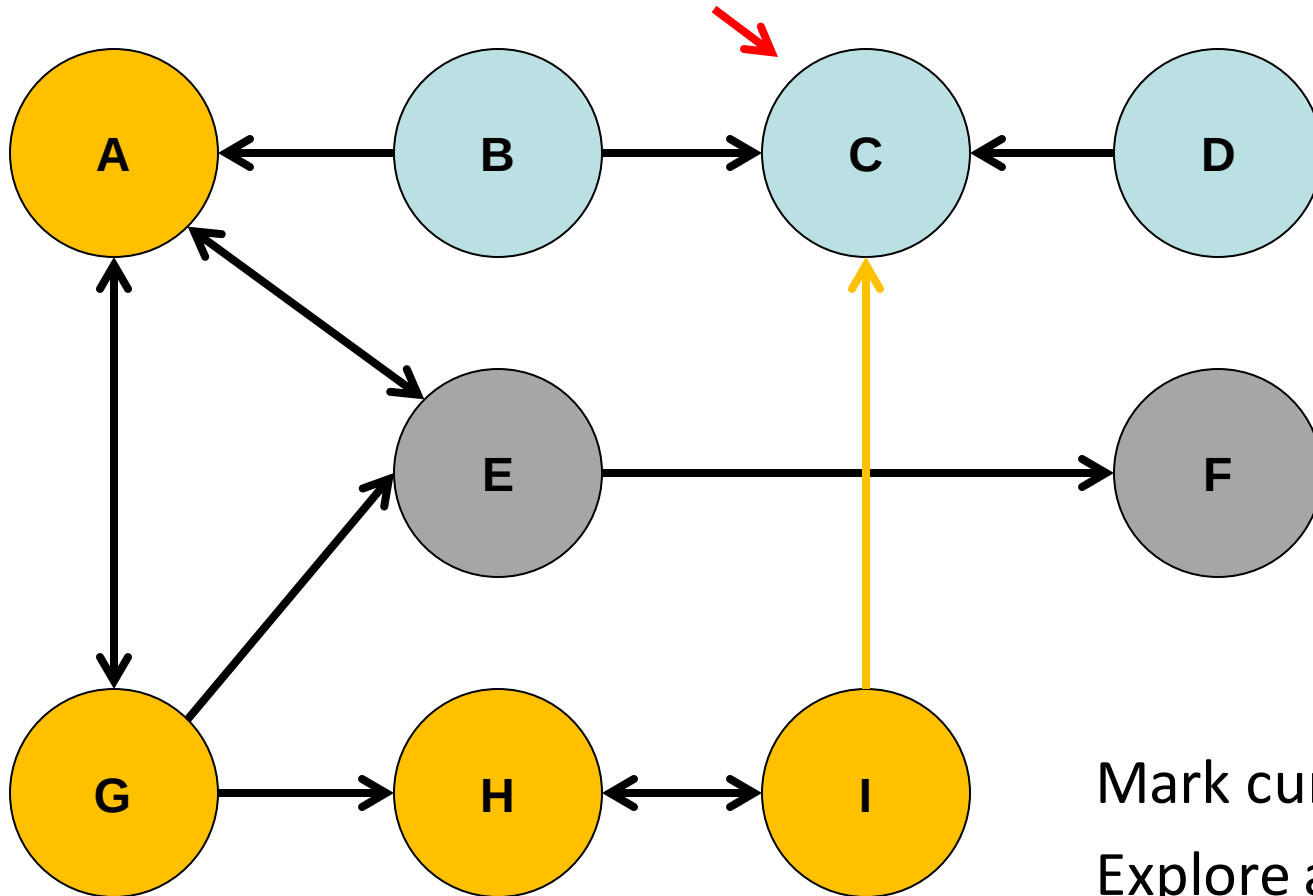
Mark current as visited
Explore all the unvisited
nodes from this node

DFS



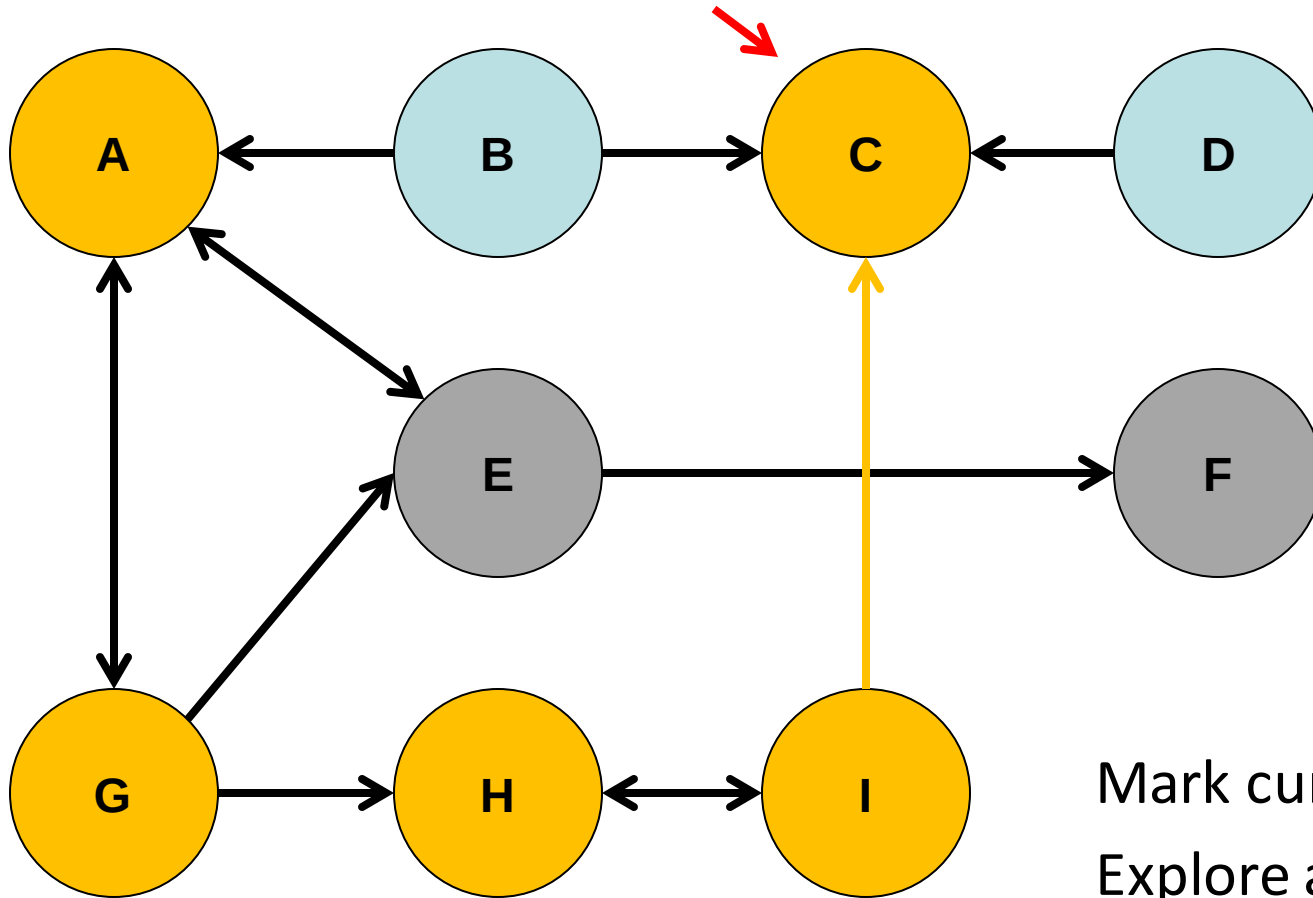
Mark current as visited
Explore all the unvisited
nodes from this node

DFS



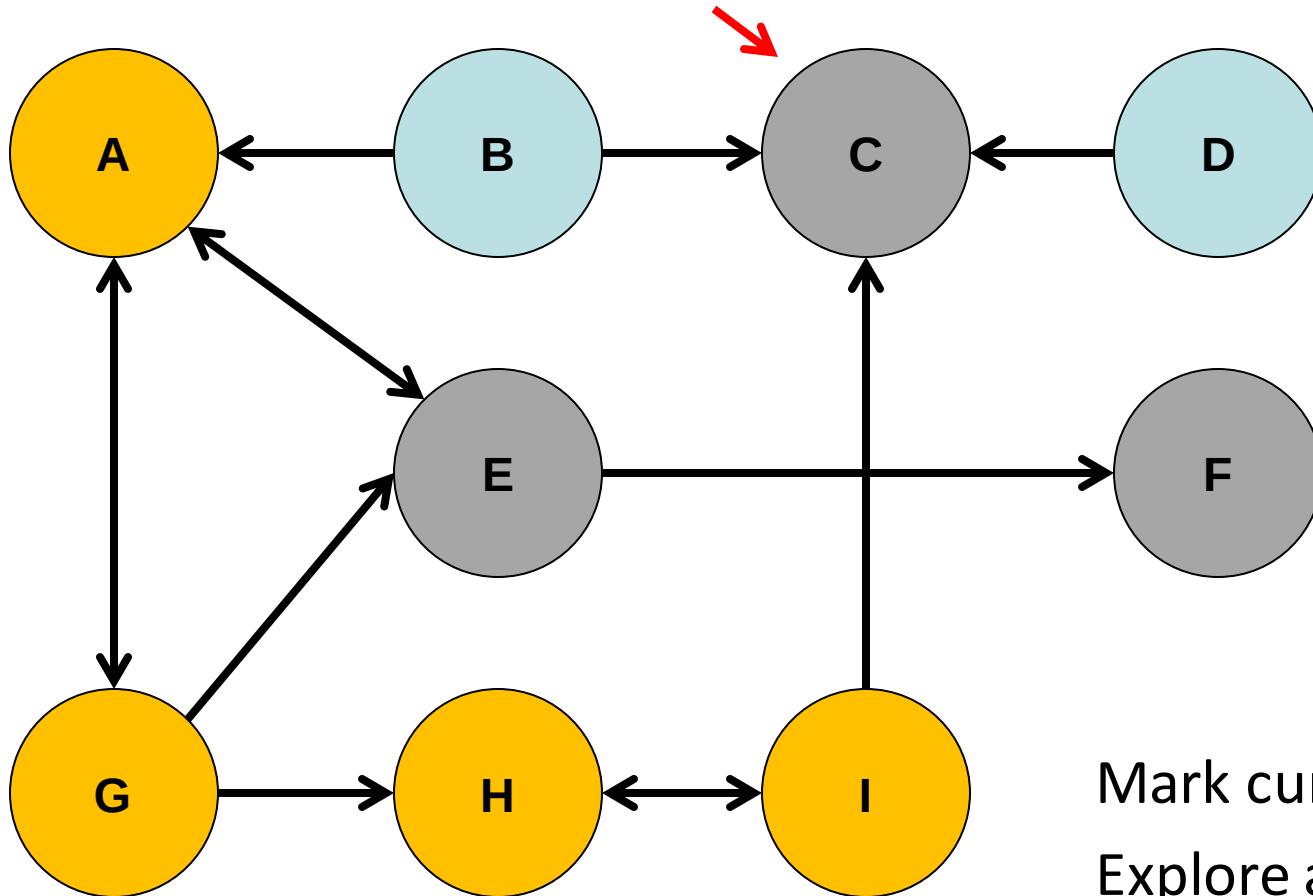
Mark current as visited
Explore all the unvisited
nodes from this node

DFS



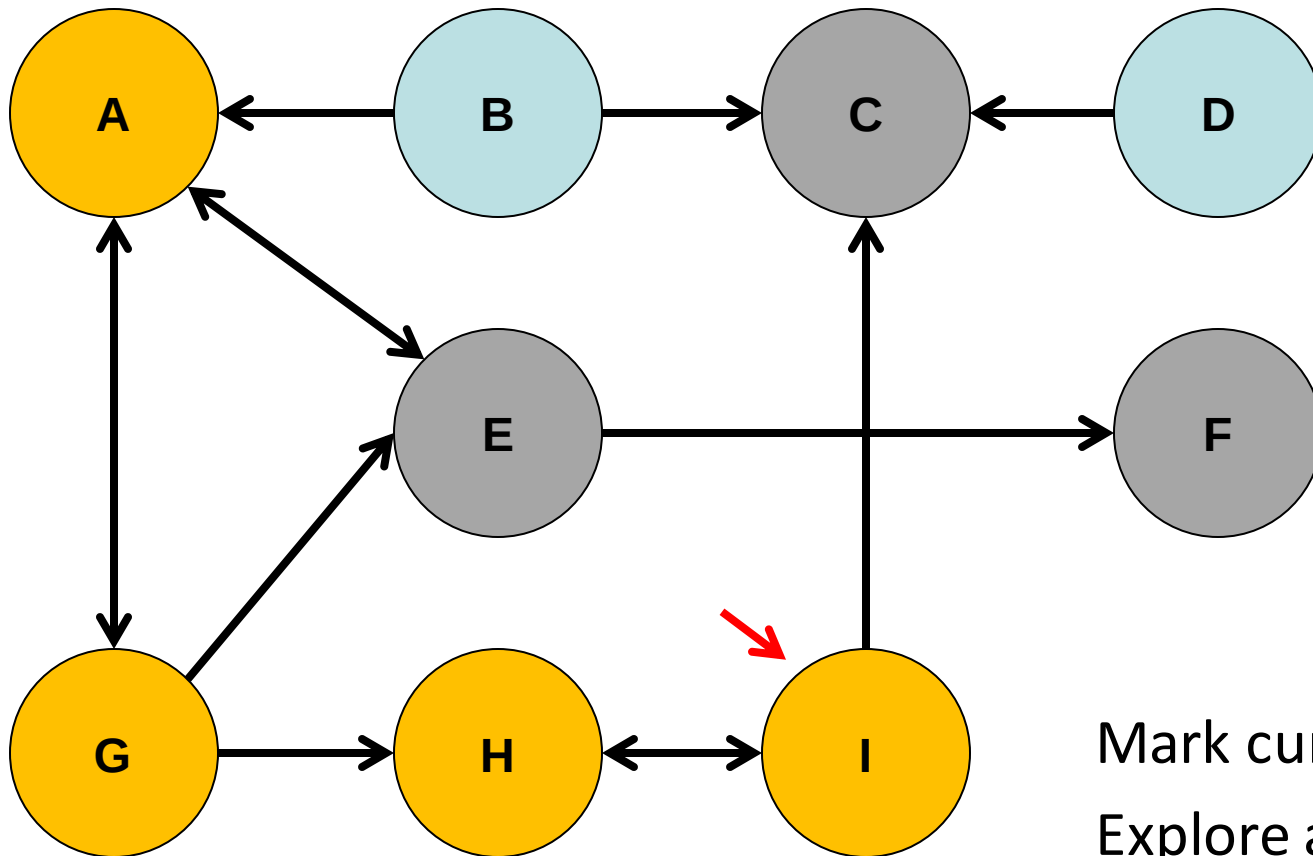
Mark current as visited
Explore all the unvisited
nodes from this node

DFS



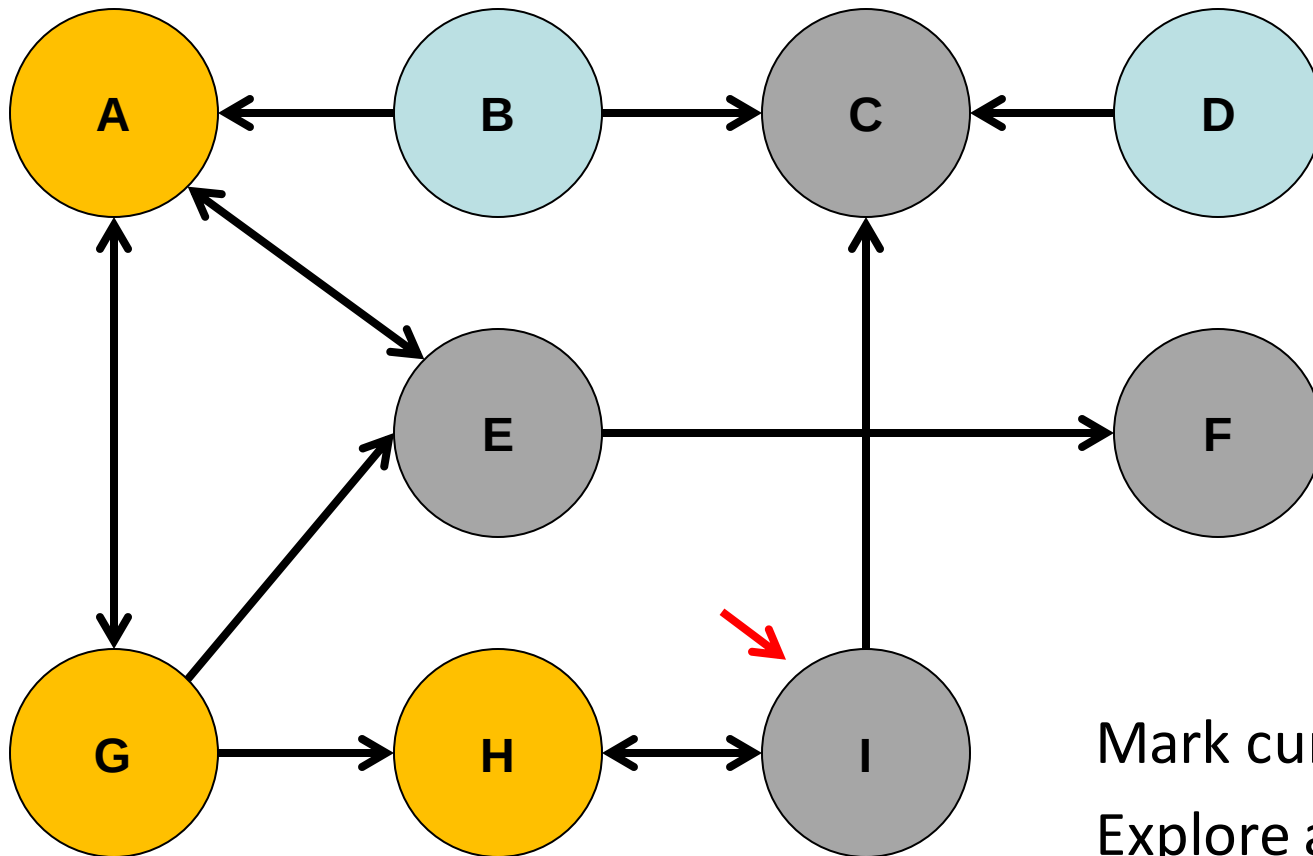
Mark current as visited
Explore all the unvisited
nodes from this node

DFS



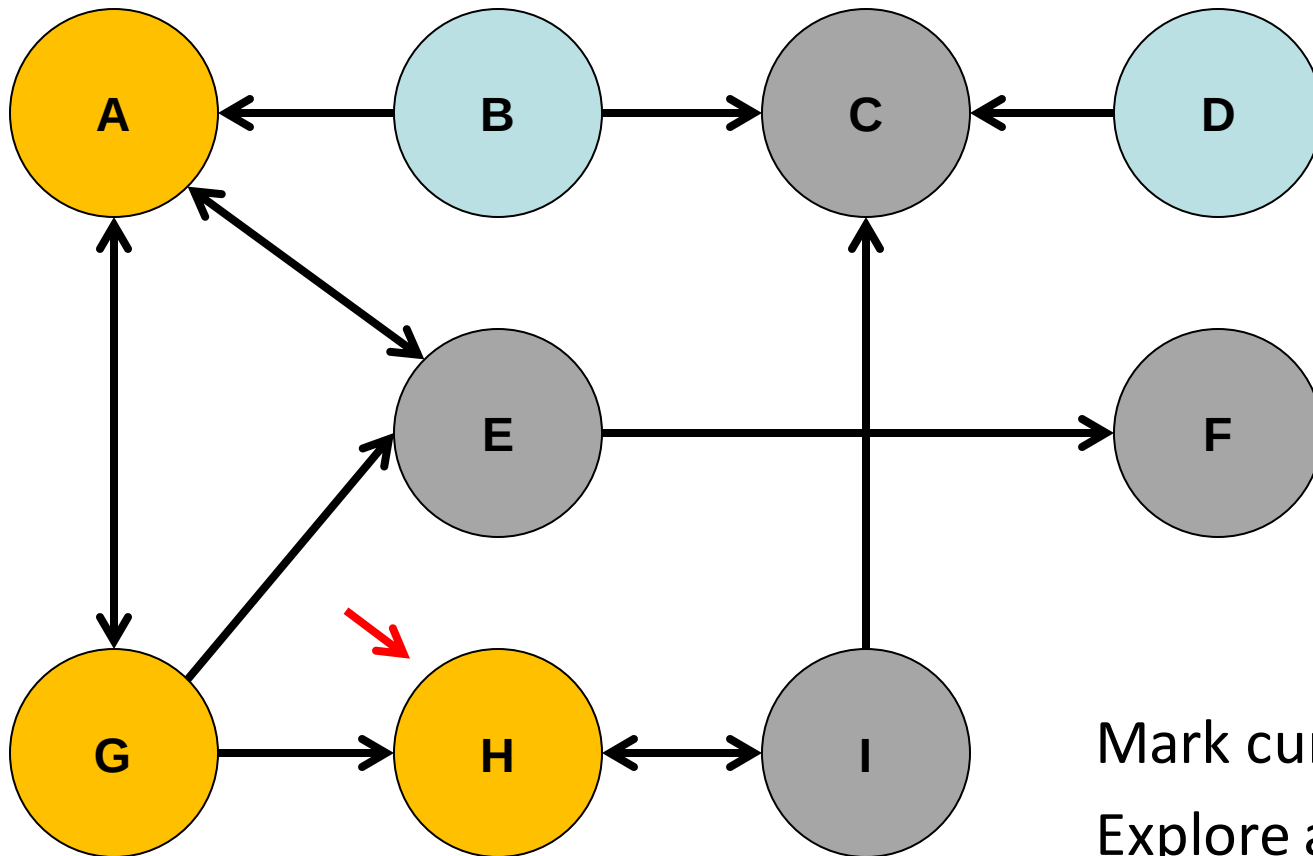
Mark current as visited
Explore all the unvisited
nodes from this node

DFS



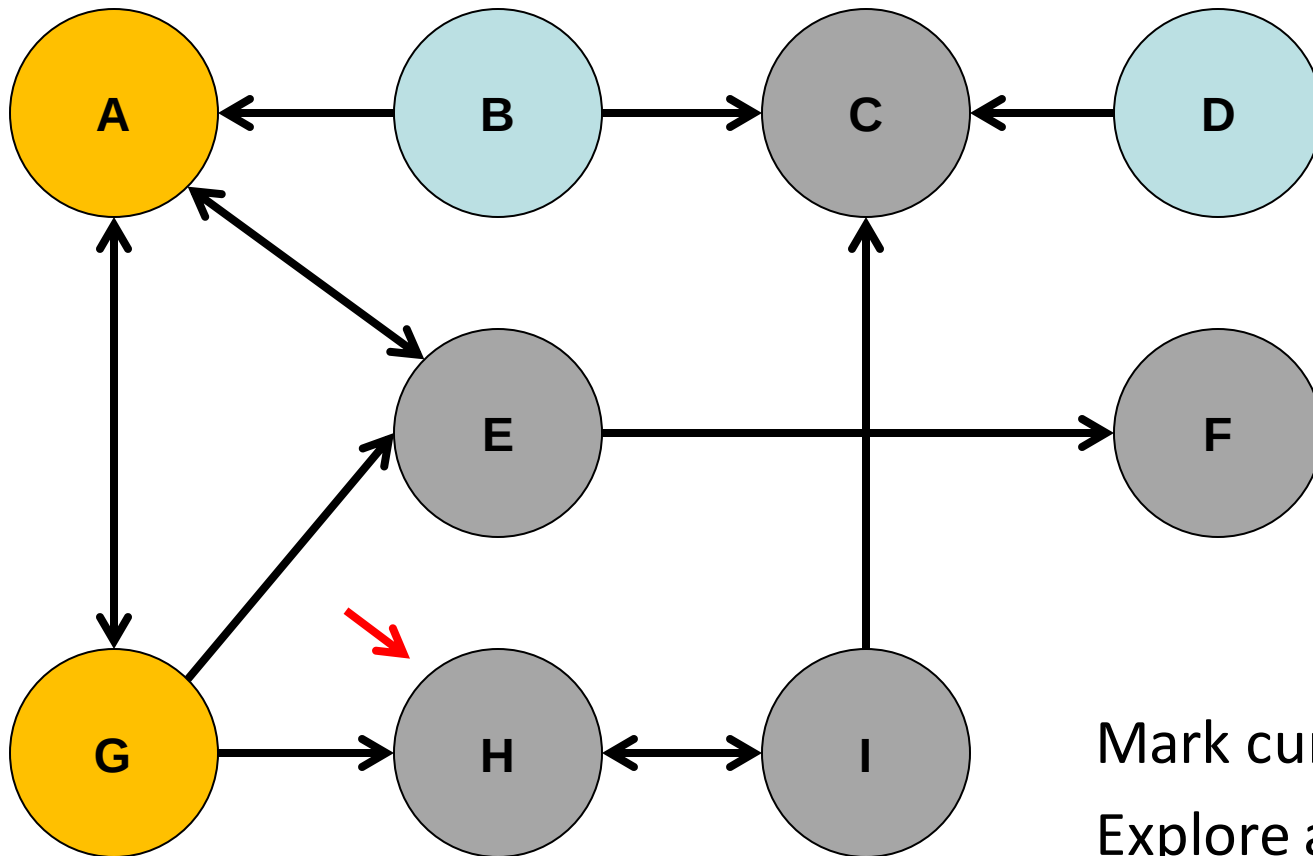
Mark current as visited
Explore all the unvisited
nodes from this node

DFS



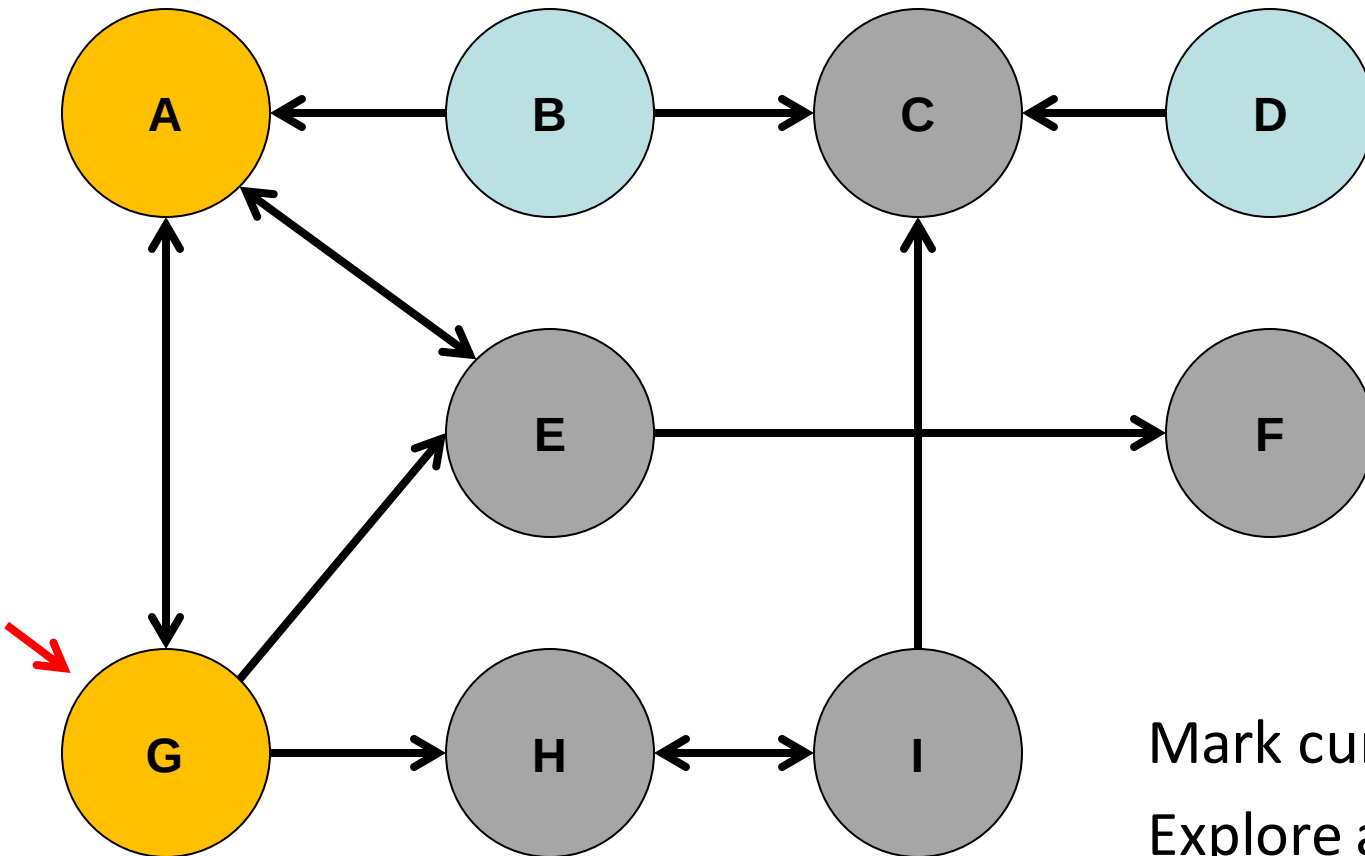
Mark current as visited
Explore all the unvisited
nodes from this node

DFS



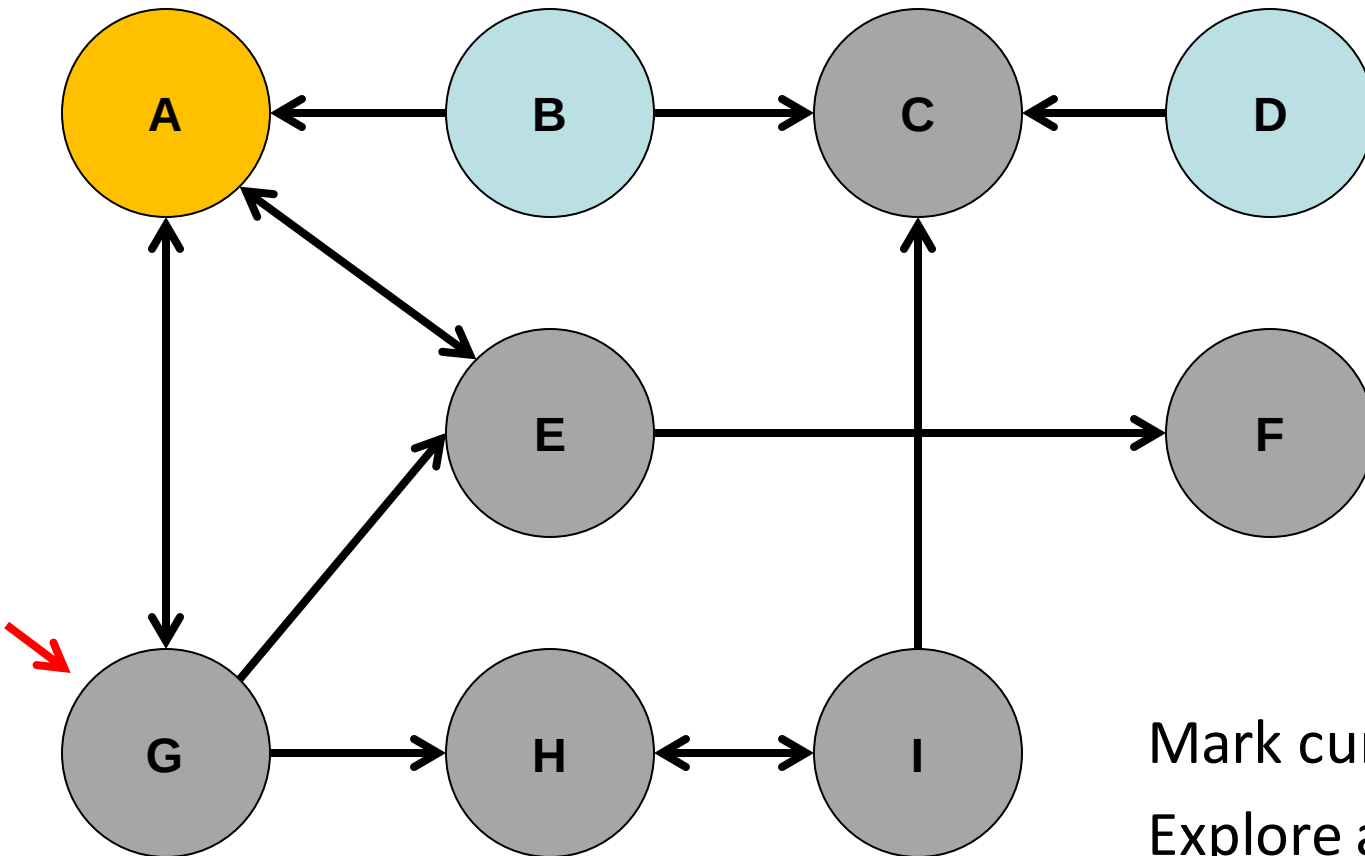
Mark current as visited
Explore all the unvisited
nodes from this node

DFS



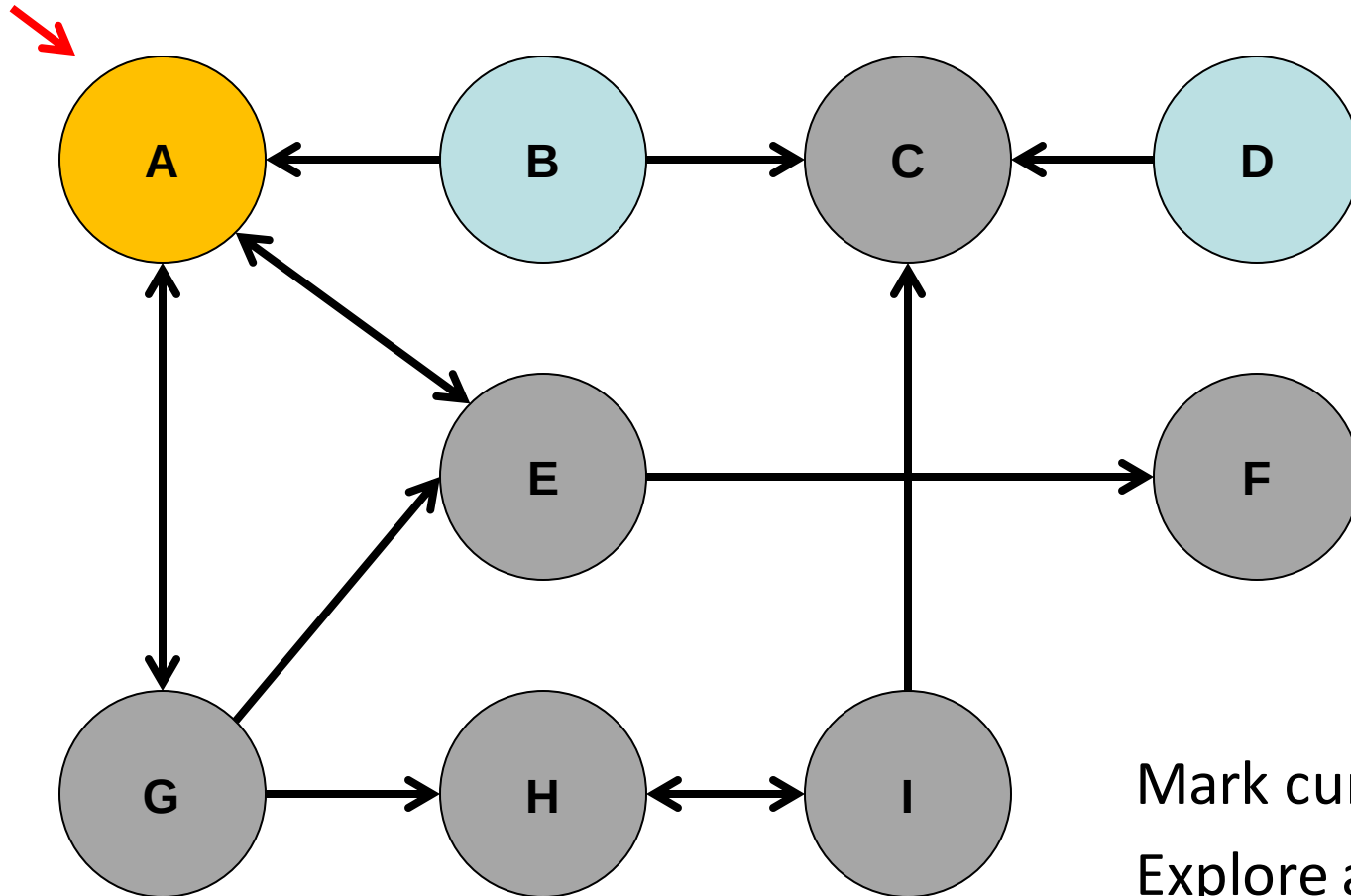
Mark current as visited
Explore all the unvisited
nodes from this node

DFS



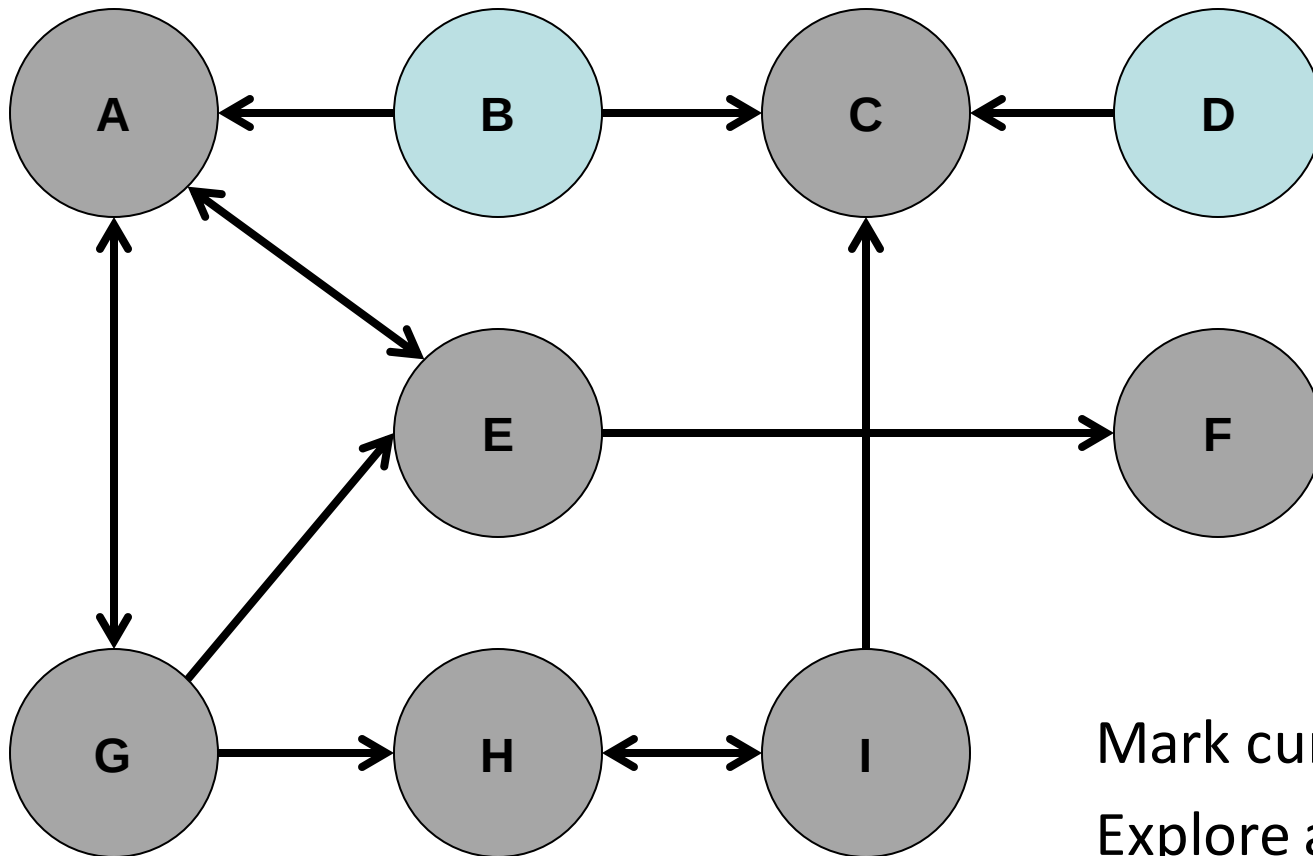
Mark current as visited
Explore all the unvisited
nodes from this node

DFS



Mark current as visited
Explore all the unvisited
nodes from this node

DFS



Mark current as visited
Explore all the unvisited
nodes from this node

DFS Details

- In an n -node, m -edge graph, takes $O(m + n)$ time with an adjacency list
 - Visit each edge once, visit each node at most once
- Pseudocode:
 dfs from v_1 :
 mark v_1 as **seen**.
 for each of v_1 's unvisited neighbors n :
 dfs(n)
- How could we modify the pseudocode to look for a specific path?

DFS that finds path

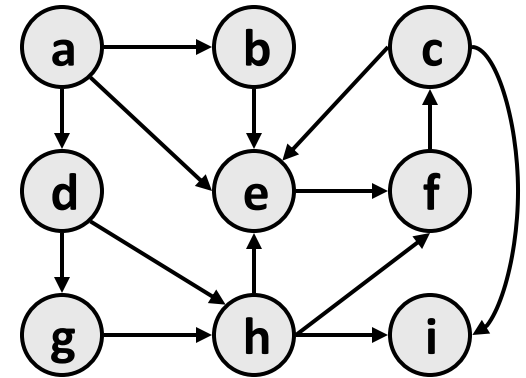
dfs from v_1 to v_2 :

mark v_1 as **visited**, and **add to path**.

perform a **dfs** from each of v_1 's
unvisited neighbors n to v_2 :

if **dfs**(n, v_2) succeeds: a path is found!

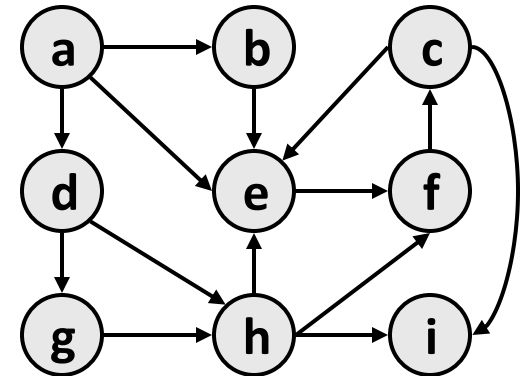
if all neighbors fail: **remove v_1 from path**.



- To retrieve the DFS path found, pass a collection parameter to each call and choose-explore-unchoose.

DFS observations

- *discovery*: DFS is guaranteed to find a path if one exists.
- *retrieval*: It is easy to retrieve exactly what the path is (the sequence of edges taken) if we find it
 - choose - explore - unchoose
- *optimality*: not optimal. DFS is guaranteed to find a path, not necessarily the best/shortest path
 - Example: $\text{dfs}(a, i)$ returns $\{a, b, e, f, c, i\}$ rather than $\{a, d, h, i\}$.



Finding *Shortest* Paths

- We can find paths between two nodes, but how can we find the **shortest** path?
 - Fewest number of steps to complete a task?
 - Least amount of edits between two words?

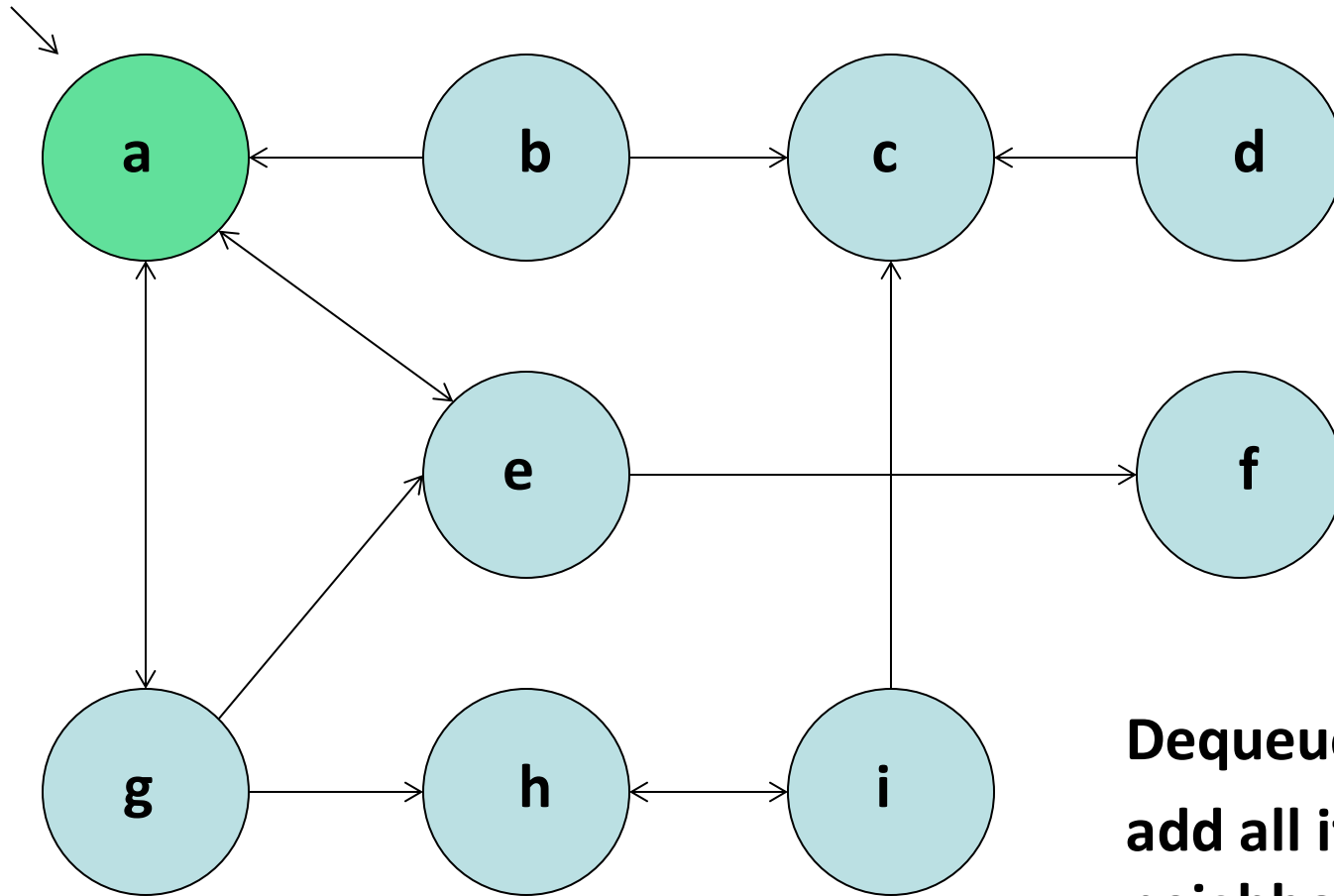
Breadth-First Search (BFS)

- Idea: processing a node involves knowing we need to visit all its neighbors (just like DFS)
- Need to keep a TODO list of nodes to process

Breadth-First Search (BFS)

- Keep a Queue of nodes as our TODO list
- Idea: dequeue a node, enqueue all its neighbors
- Still will return the same nodes as reachable, just might have shorter paths

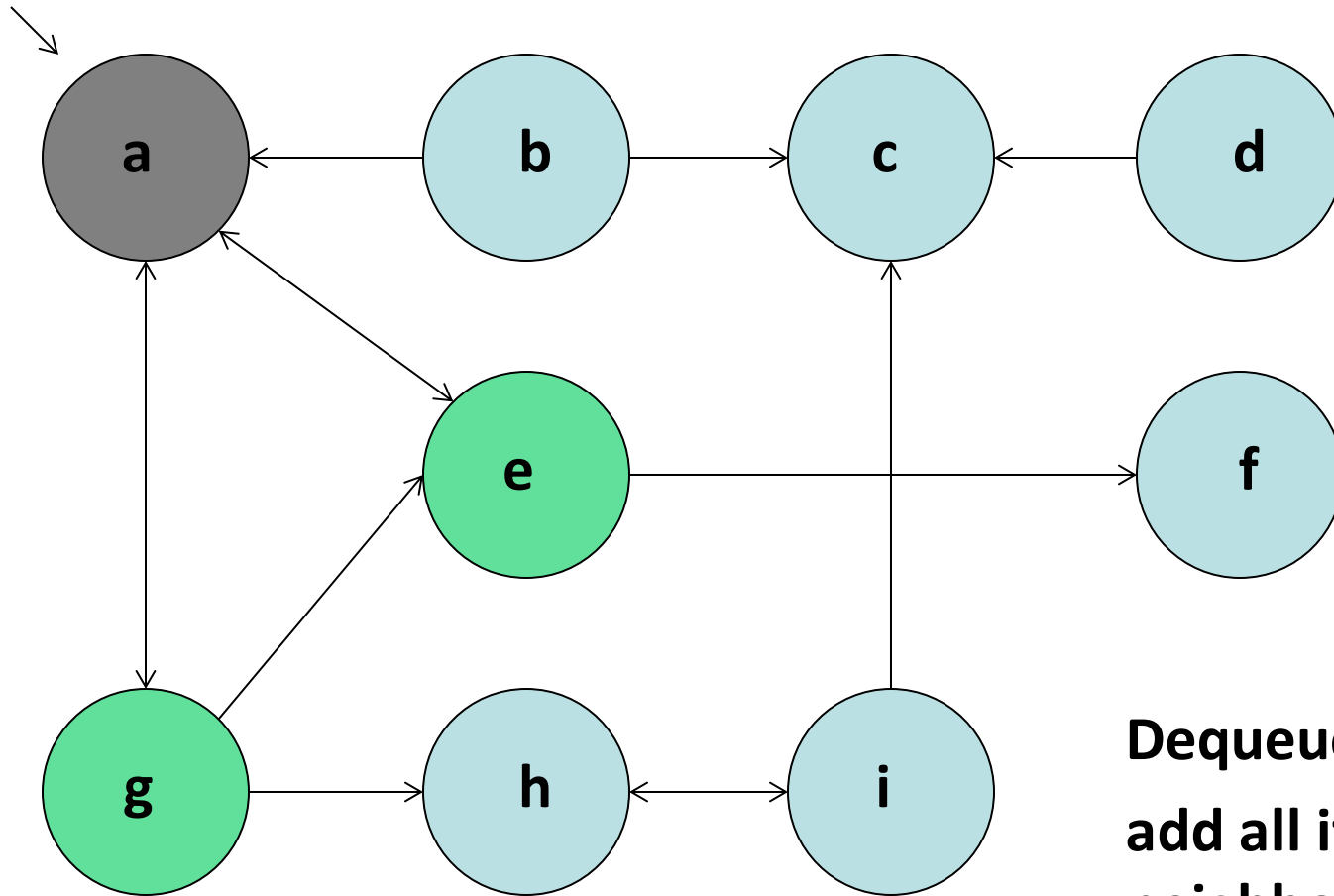
BFS



**Dequeue a node
add all its unseen
neighbors to the queue**

queue: a

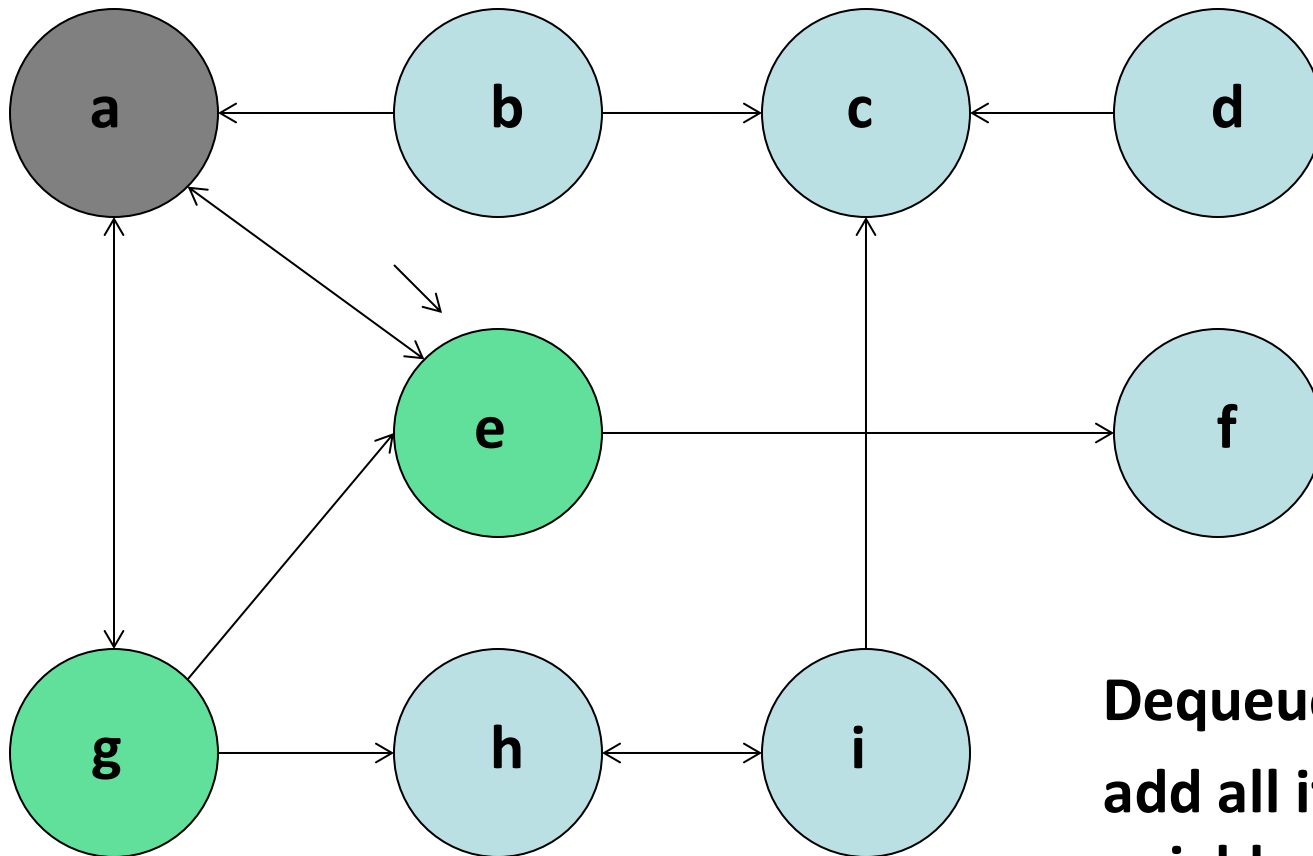
BFS



**Dequeue a node
add all its unseen
neighbors to the queue**

queue: e, g

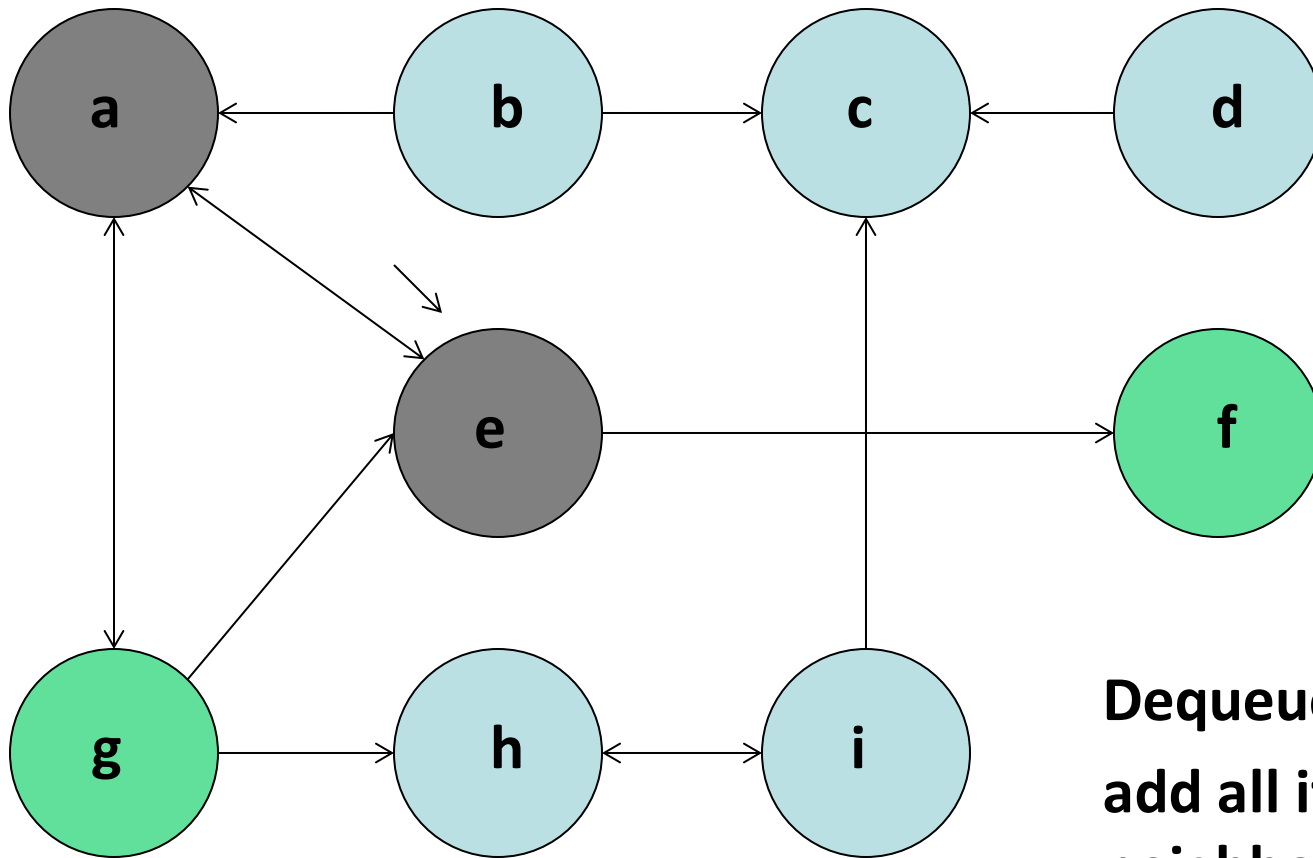
BFS



**Dequeue a node
add all its unseen
neighbors to the queue**

queue: e, g

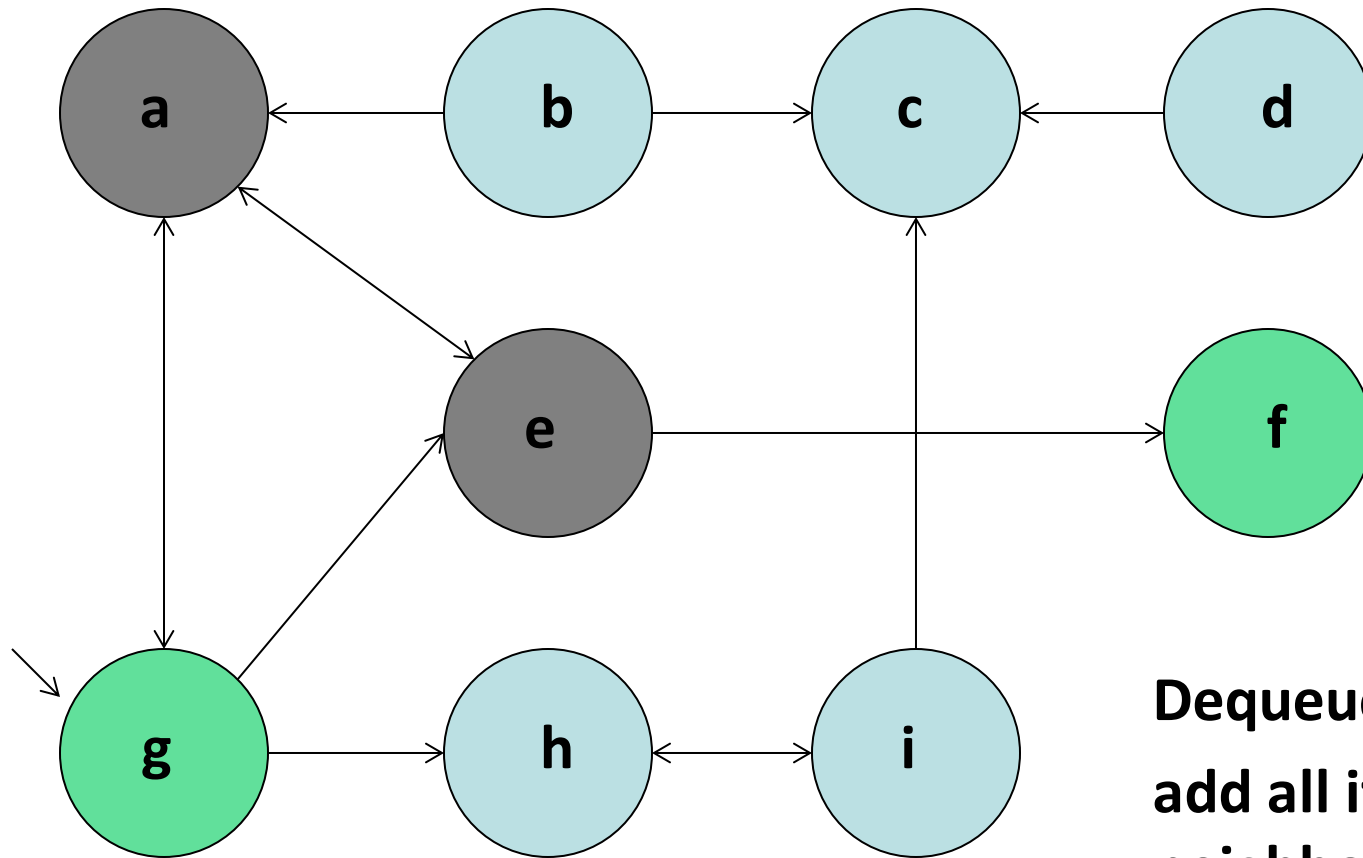
BFS



queue: g, f

**Dequeue a node
add all its unseen
neighbors to the queue**

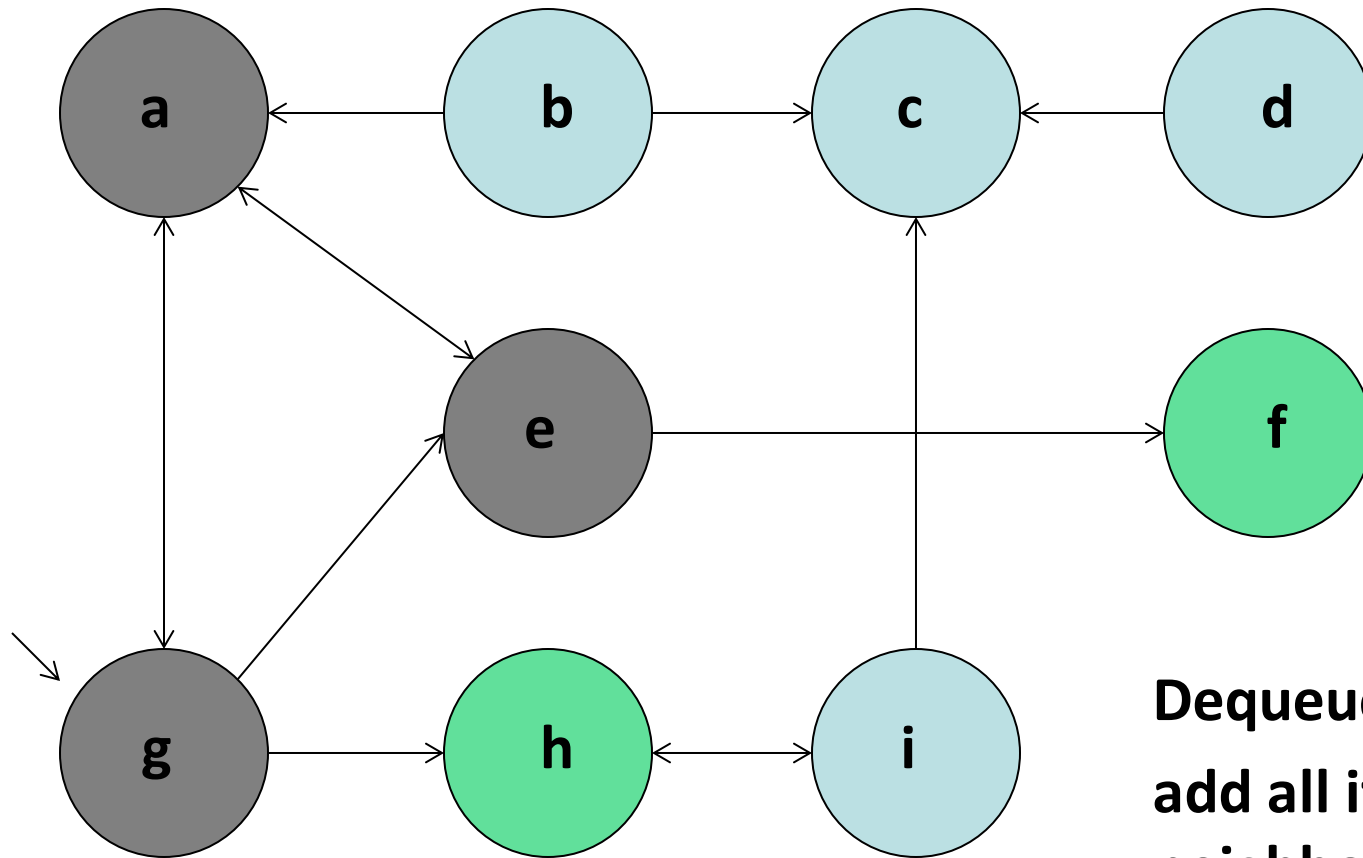
BFS



**Dequeue a node
add all its unseen
neighbors to the queue**

queue: g, f

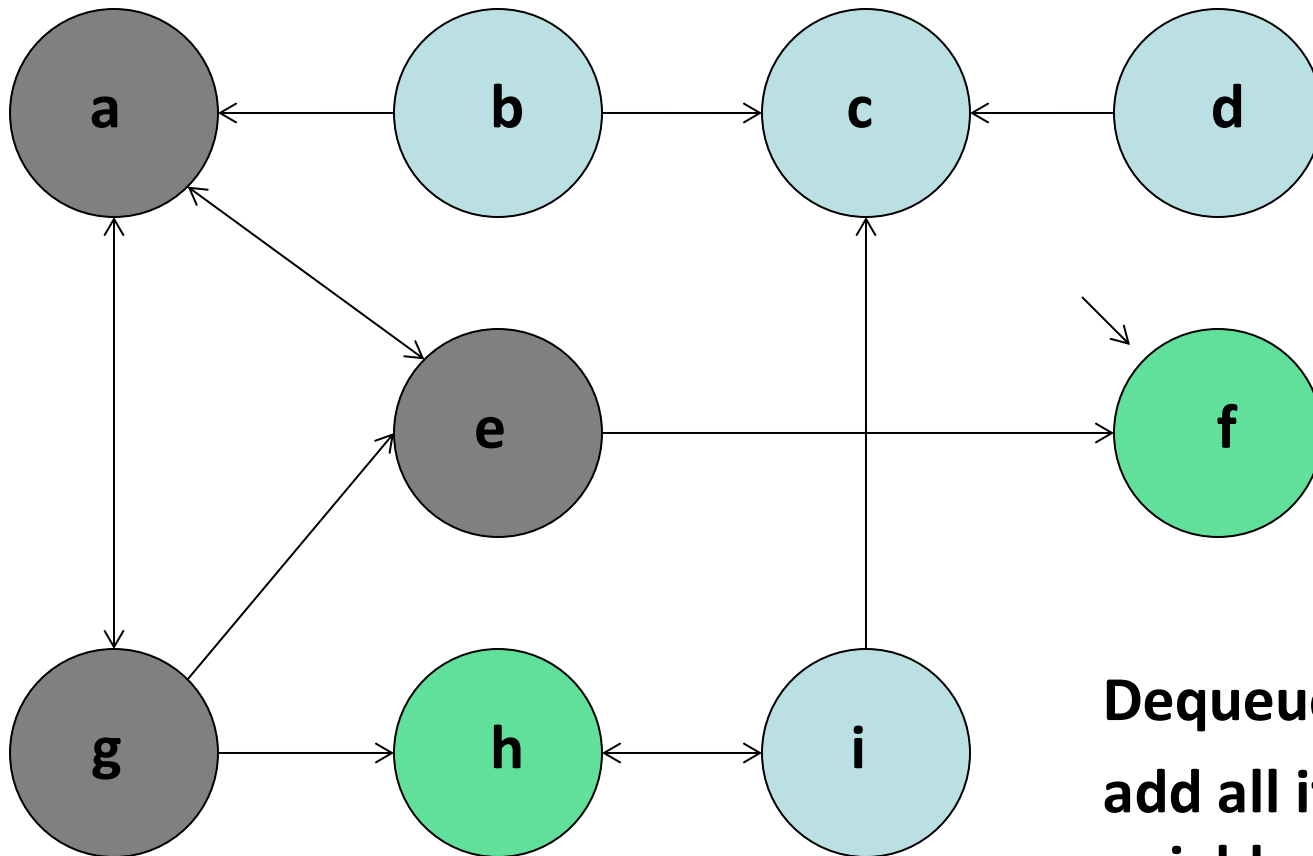
BFS



**Dequeue a node
add all its unseen
neighbors to the queue**

queue: f, h

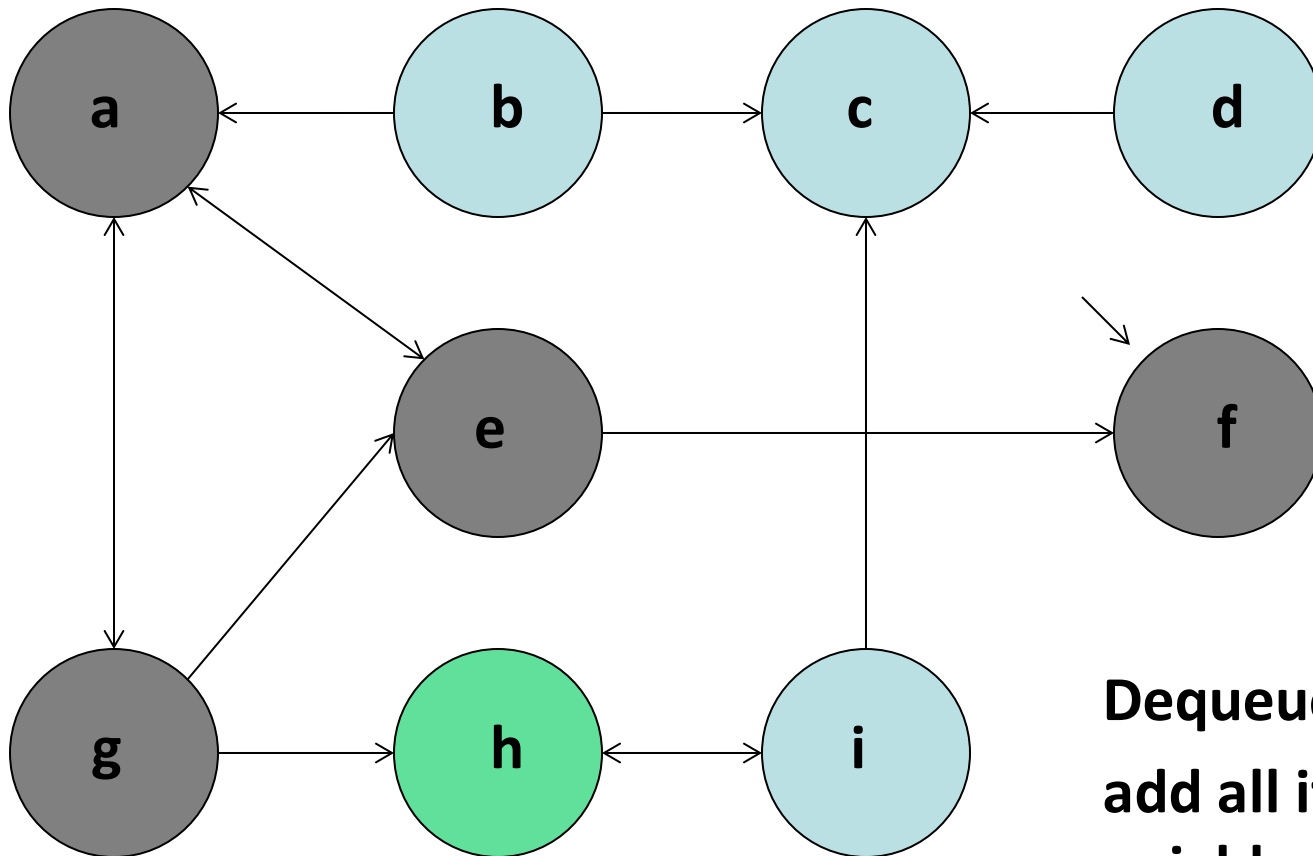
BFS



**Dequeue a node
add all its unseen
neighbors to the queue**

queue: f, h

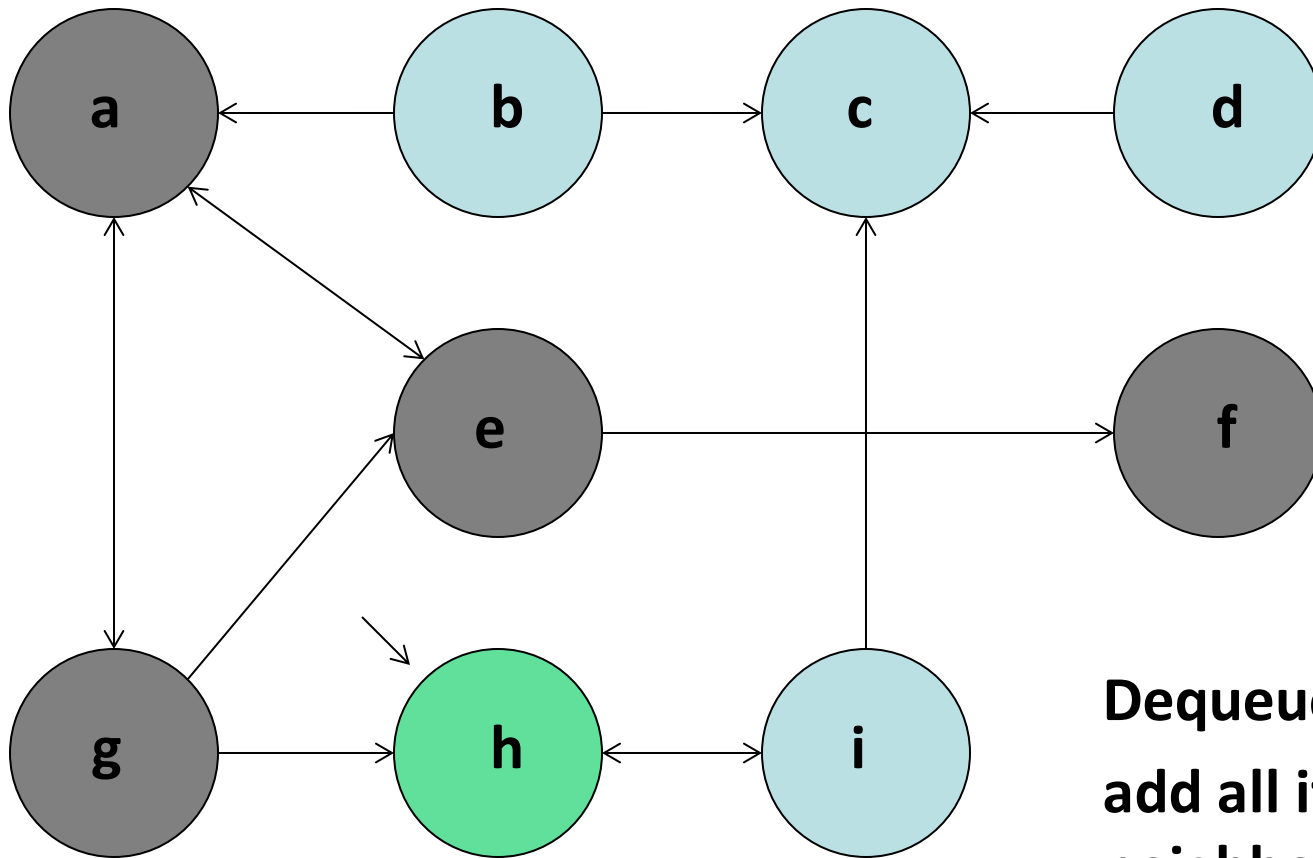
BFS



**Dequeue a node
add all its unseen
neighbors to the queue**

queue: h

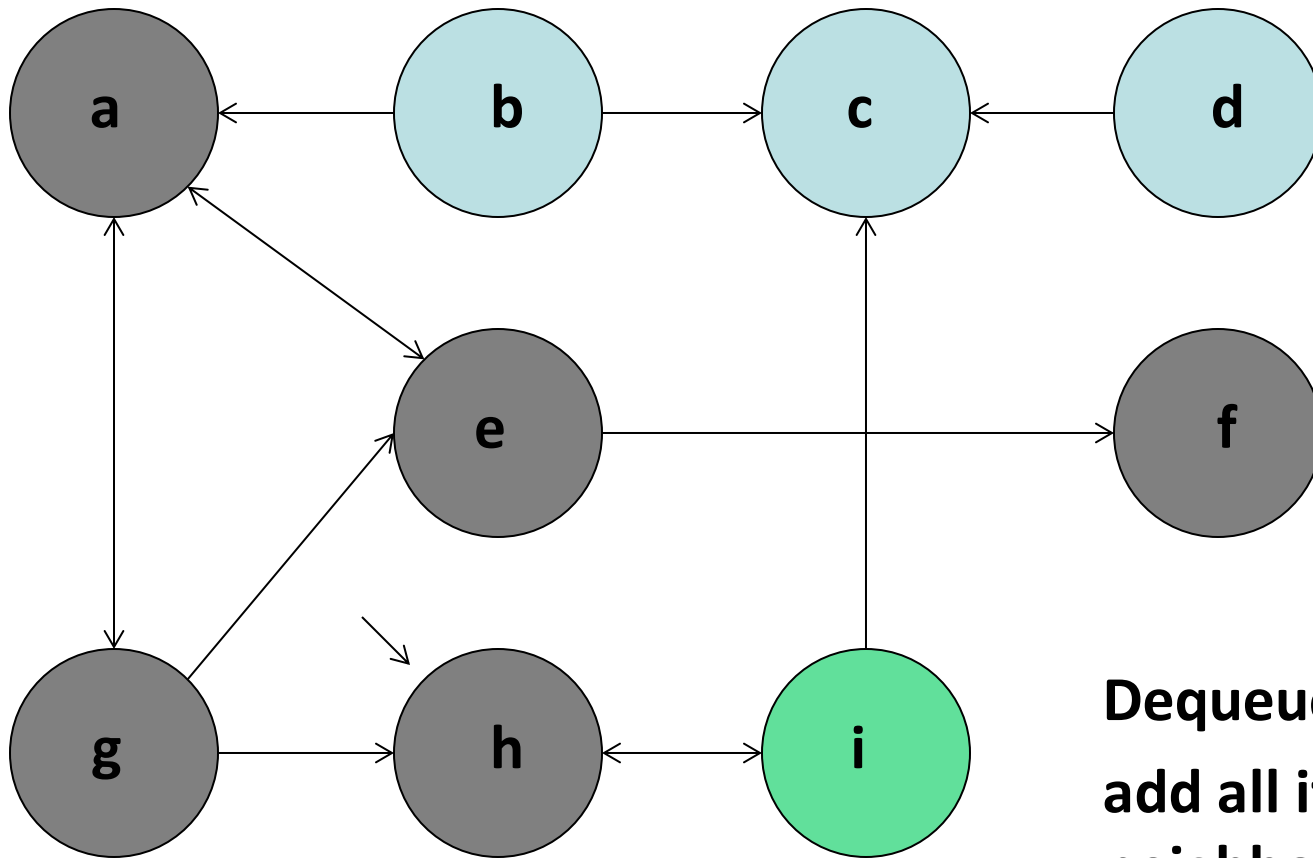
BFS



**Dequeue a node
add all its unseen
neighbors to the queue**

queue: h

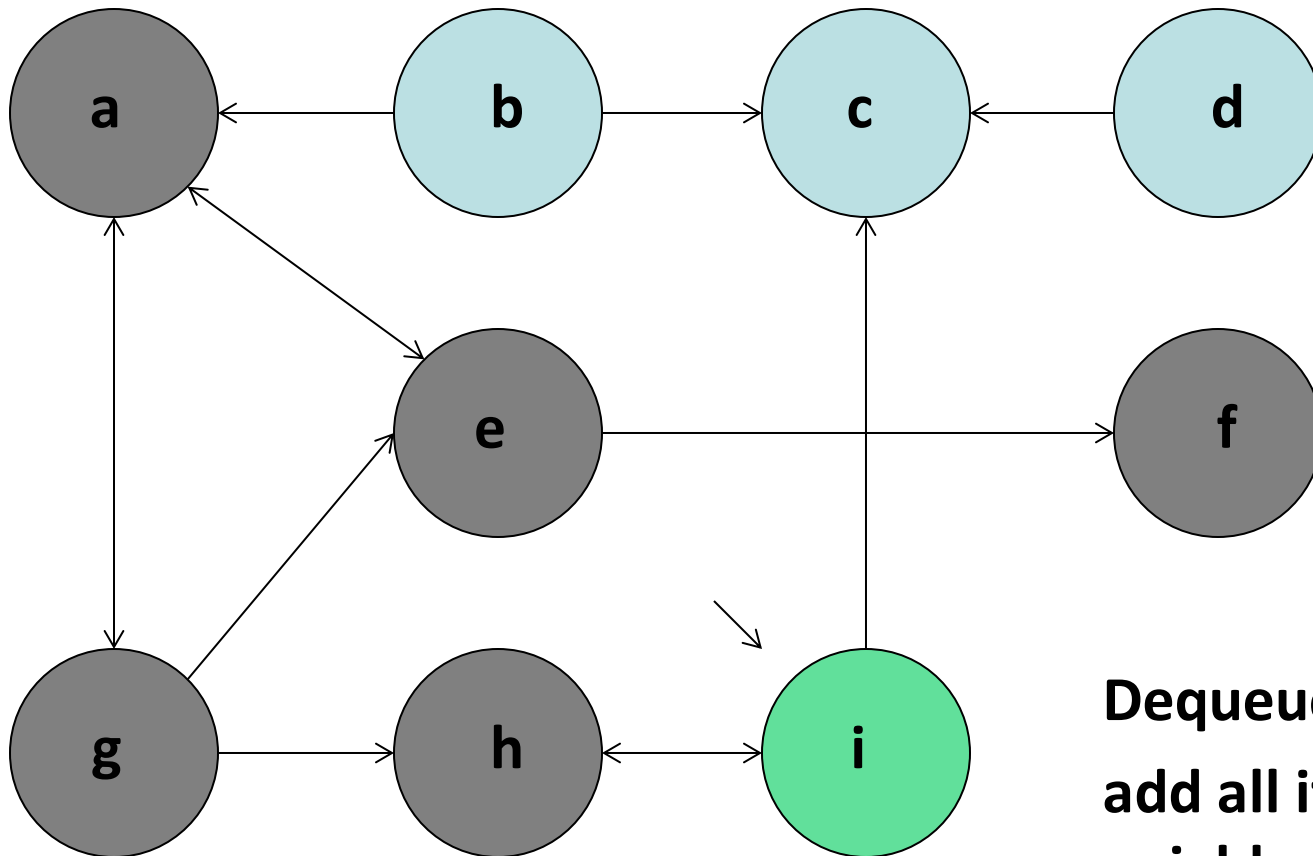
BFS



**Dequeue a node
add all its unseen
neighbors to the queue**

queue: i

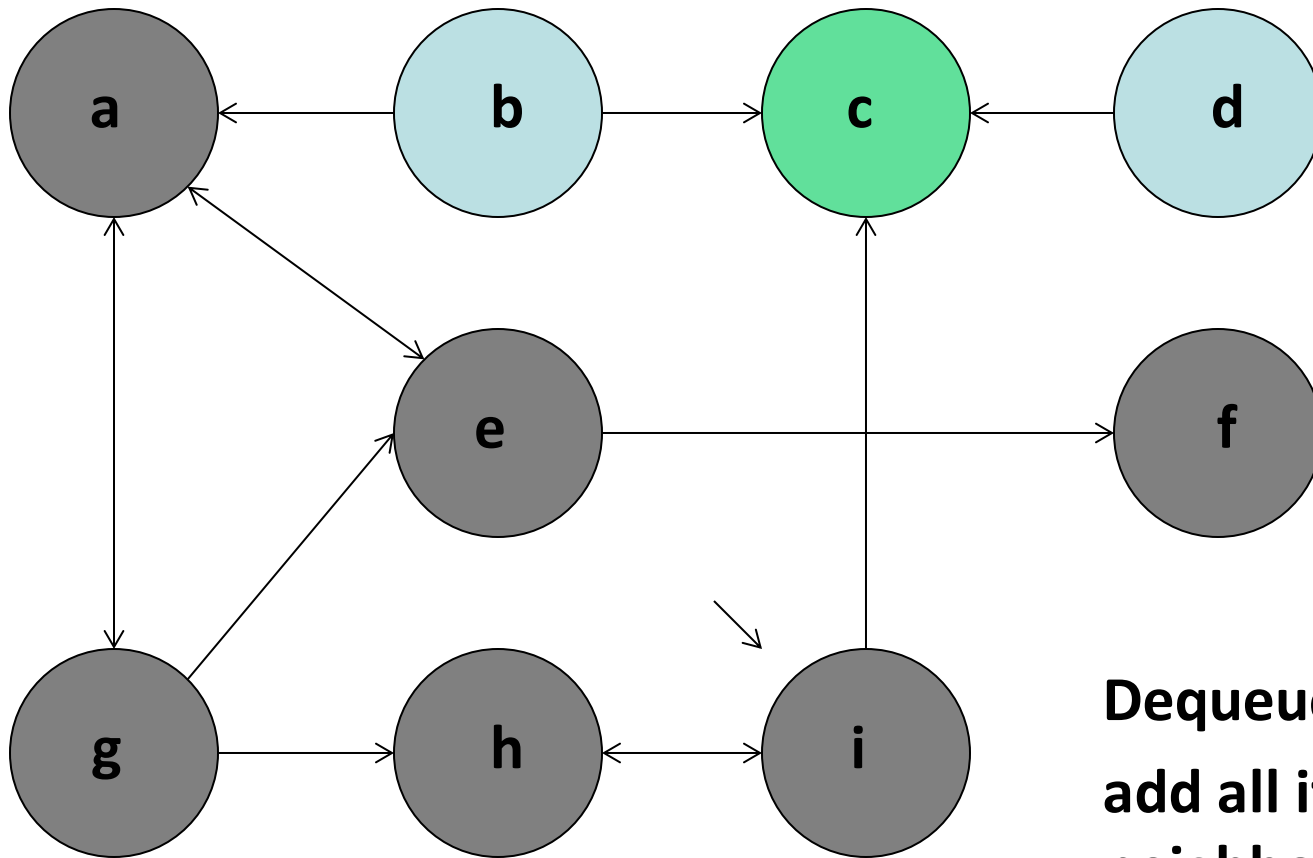
BFS



**Dequeue a node
add all its unseen
neighbors to the queue**

queue: i

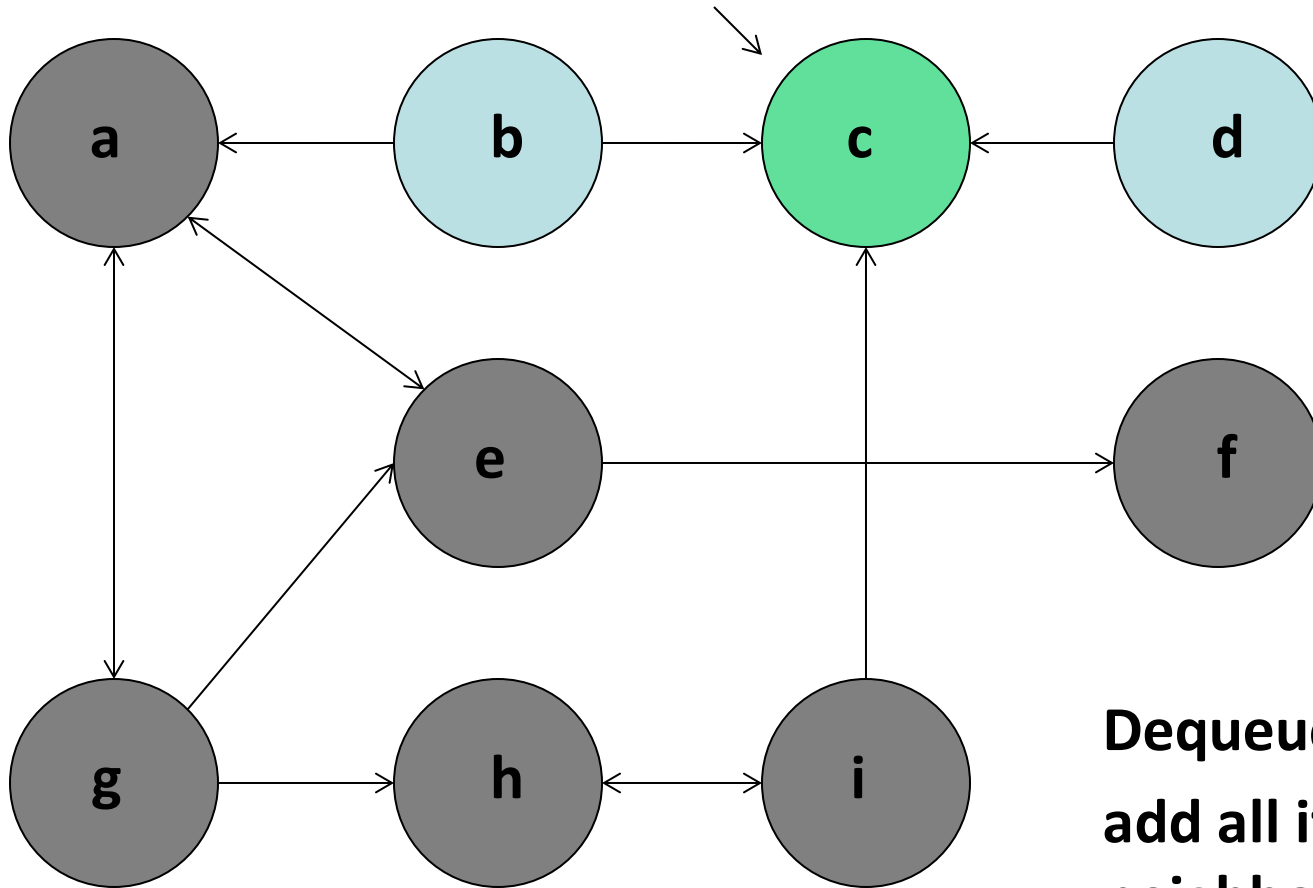
BFS



**Dequeue a node
add all its unseen
neighbors to the queue**

queue: c

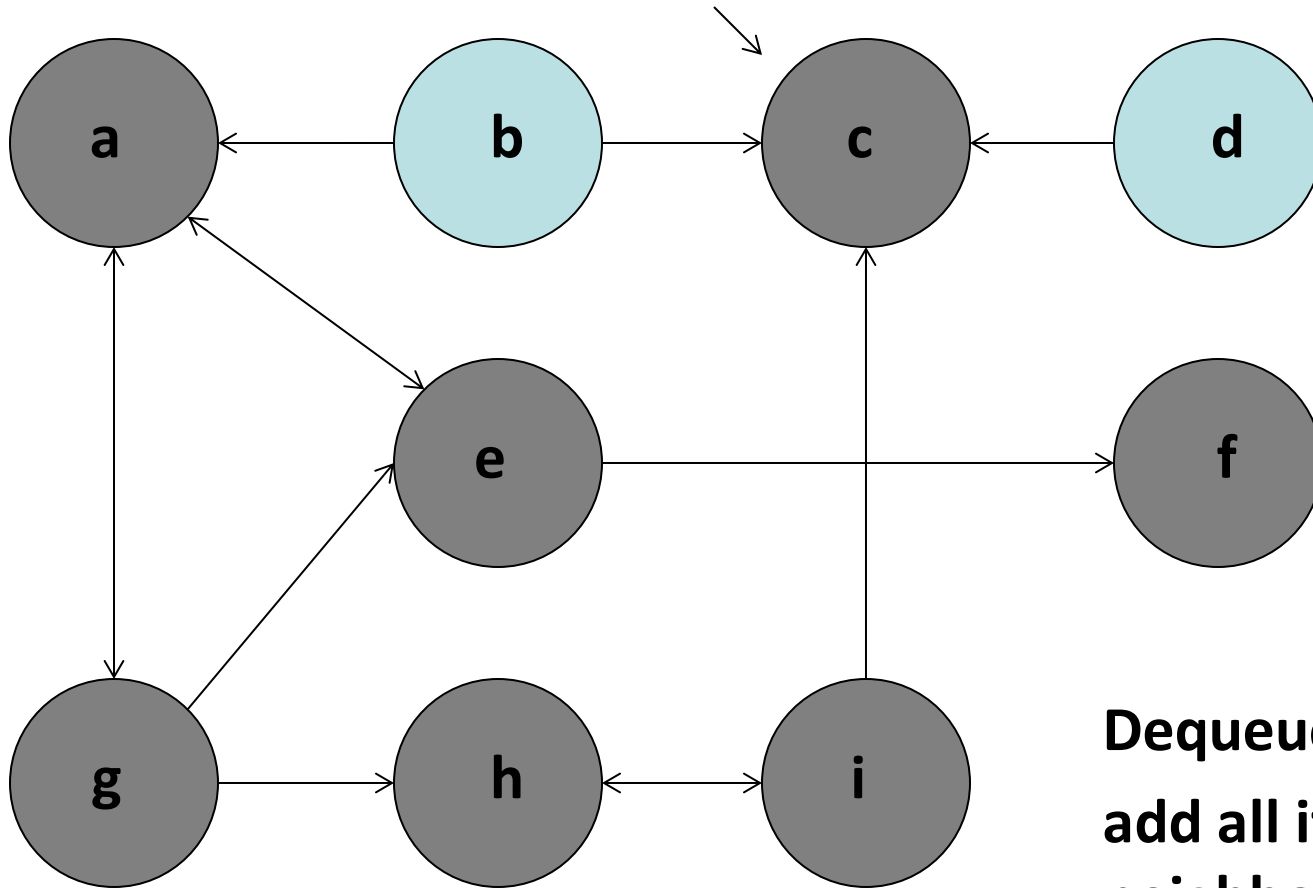
BFS



**Dequeue a node
add all its unseen
neighbors to the queue**

queue: c

BFS



**Dequeue a node
add all its unseen
neighbors to the queue**

queue: c

BFS Details

- In an n -node, m -edge graph, takes $O(m + n)$ time with an adjacency list
 - Visit each edge once, visit each node at most once

bfs from v_1 to v_2 :

create a *queue* of vertexes to visit,
initially storing just v_1 .

mark v_1 as **visited**.

while *queue* is not empty and v_2 is not seen:

dequeue a vertex v from it,

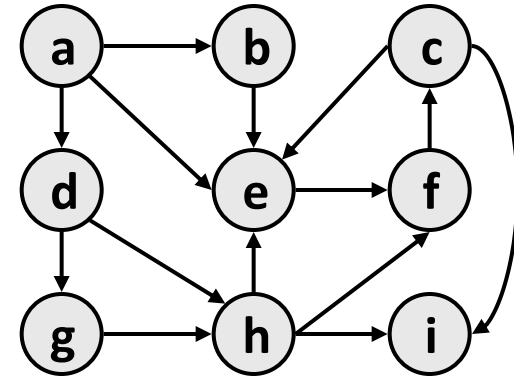
mark that vertex v as **visited**,

and add each unvisited neighbor n of v to the *queue*.

- How could we modify the pseudocode to look for a specific path?

BFS observations

- *optimality*:
 - always finds the shortest path (fewest edges).
 - in unweighted graphs, finds optimal cost path.
 - In weighted graphs, *not* always optimal cost.
- *retrieval*: harder to reconstruct the actual sequence of vertices or edges in the path once you find it
 - conceptually, BFS is exploring many possible paths in parallel, so it's not easy to store a path array/list in progress
 - solution: We can keep track of the path by storing predecessors for each vertex (each vertex can store a reference to a *previous* vertex).
- DFS uses less memory than BFS, easier to reconstruct the path once found; but DFS does not always find shortest path. BFS does.



BFS that finds path

bfs from v_1 to v_2 :

create a *queue* of vertexes to visit,
initially storing just v_1 .
mark v_1 as **visited**.

while *queue* is not empty and v_2 is not seen:

dequeue a vertex v from it,

mark that vertex v as **visited**,

and add each unvisited neighbor n of v to the *queue*,

while setting n 's **previous** to v .

